

MADAPORC

Mini soutenance S4 (2024-2025)

Système Intelligent de Suivi, de Traçabilité et d'Aide à la Décision Reproductive pour l'Élevage Porcin

Présenté par :

Groupe 16-A

Encadreur : **Baovola RAJAONARISON**

Membres du jury:

Les membres du groupe 16-A:

ANDRIAMARONDRAIBE Sambatra Mickaella (chef du projet) ETU004346
RAMAROMANANA Tsanta Mahery Finaritra R. (chef interime) ETU004590
RAJOELISON Andriamaholy Aina Manoa ETU004079
ANDRIANANTENAINA Noah ETU004326
ANDRIAMIADANARIVO Fanilo Tsitohaina ETU004043
ANDRIANTSIFERANA Nomena Sarobidy ETU004349
RANOMENJANAHARY Mandresy Ally ETU004342
RAJAOSON Ny Andry Miaro ETU004176
RAKOTOARIMANANA Davida Fiderana ETU004037

SOMMAIRE

Table des matières

PARTIE 1: CADRE DU PROJET ET BESOINS.....	3
1) Présentation du projet MADAPORC:.....	3
2) Cahier des charges fonctionnel - Madaporc.....	4
2.1) Sécurité.....	4
2.2) Gestion des lots.....	4
2.3) Gestion des pesées.....	4
2.4) Gestion de la reproduction.....	5
2.5) Analyse.....	5
2.6) Gestion sanitaire.....	5
2.7) Gestion des stocks.....	5
2.8) Gestion commerciale et financière.....	6
2.9) Tableau de bord.....	6
2.10) Import / Export.....	6
PARTIE 2: CHOIX TECHNIQUE ET ARCHITECTURE.....	7
1) Introduction.....	7
2) Choix technologique justifié.....	7
2.1) Backend.....	7
2.2) Base de données.....	8
2.3) Moteur de templates.....	9
3) Comparaison des technologies – Projet Madaporc.....	9
3.1) Backend : Spring Boot vs Laravel.....	9
3.2) Java 17 vs PHP.....	10
3.3) PostgreSQL vs Oracle Database.....	10
3.4) JSP vs Thymeleaf.....	11
4) Architecture globale du projet:.....	12
4.1) Couche de présentation (Views):.....	13
4.2) Couche Contrôleur (Controller):.....	14
4.3) Couche Service:.....	15
4.4) Couche Model:.....	16
4.5) Couche DTO:.....	17
5) WorkFlow du projet:.....	18
6) Architecture technique du projet:.....	20
PARTIE 3: CONCEPTION DE LA BASE DE DONNÉES.....	21
1) a.Module Gestion des users:.....	21
b.MCD:.....	21
2) a.Module Gestion des porcs:.....	22
b.MCD:.....	24
3) a.Module reproduction:.....	25
b.MCD:.....	28
4) a.Module Analyse de reproduction:.....	29
b.MCD:.....	32
5) a.Module sanitaire:.....	32
b.MCD:.....	35
6) a.Module vaccinations:.....	36

b.MCD:.....	38
7) a.Module Stock Alimentaire:.....	38
b.MCD:.....	39
8) a.Module commercial (clients / ventes):.....	40
b.MCD :.....	42
9) a.Module dépenses:.....	43
b.MCD:.....	43
10) a.Module Alertes:.....	44
b.MCD:.....	48
11) a.Module Import-Export:.....	48
b.MCD:.....	49
PARTIE 4: PILOTAGE DU PROJET ET PERSPECTIVES.....	50
1) Organisation du projet:.....	50
1.1) Userflow:.....	50
1.2) Planning:.....	61
1.3) Budget:.....	63
2) Perspectives du projet:.....	64
. Évolutivité technologique (Scalabilité).....	64
. Intelligence Artificielle et Aide à la Décision.....	64
. Intégration Écosystémique.....	64
Conclusion:.....	65

Liste des schémas et tableaux:

Tableau 1: Présentation technologie du projet côté backend

Tableau 2: Présentation technologie du projet côté frontend

Tableau 3: Présentation technologie du projet côté user

Tableau 4: Benchmarking technologique côté backend

Tableau 5: Benchmarking technologique côté frontend

Tableau 6: Benchmarking technologie du projet côté user

Tableau 7: Analyse du budget de l'entreprise

Tableau 8: Synthèse de présence des membres de l'équipe

Schéma 1-6: Architecture du projet

Schéma 7: Workflow du projet

Schéma 8: Architecture technique du projet

Schéma 9-19: MCD

Schéma 20-29: Planning (Gantt)

PARTIE 1: CADRE DU PROJET ET BESOINS

1) Présentation du projet MADAPORC:

Le marché de la vente de porcs présente un fort potentiel, puisque Madagascar enregistre une forte consommation de viande porcine. Cependant, beaucoup de personnes n'entrent pas dans ce secteur en raison des exigences élevées liées à l'élevage des porcs. En effet, la gestion de l'élevage qui englobe la reproduction des bêtes, les vaccinations, les ventes et toutes les dépenses financières se fait encore à la main.

Pour remédier à cela, le projet MADAPORC a été créé. MADAPORC est une application web destinée à la gestion moderne d'une exploitation porcine. Au-delà d'un simple outil d'enregistrement administratif, le système intègre un module d'analyse automatisé de la reproduction et du suivi sanitaire. En centralisant ces données, l'application permet d'identifier les reproducteurs les plus performants et d'anticiper les réformes. L'objectif final est de transformer ces données brutes en indicateurs fiables d'aide à la décision afin d'optimiser la productivité et la rentabilité de l'exploitation.

2) Cahier des charges fonctionnel - Madaporc

2.1) Sécurité

Rôles concernés :

- Administrateur
- Employé

Règles de gestion :

- Chaque utilisateur possède un identifiant unique.
- Seul l'administrateur gère les utilisateurs.
- Authentification obligatoire.

Critères de validation :

- Nom d'utilisateur unique.
- Mot de passe obligatoire.
- Rôle ADMIN ou EMPLOYE.

2.2) Gestion des lots

Rôles concernés :

- Administrateur
- Employé

Règles de gestion :

- Chaque lot possède un code unique.
- Un lot contient des animaux de même sexe.
- Effectif mis à jour automatiquement.

Critères de validation :

- Code obligatoire.
- Effectif > 0.
- Race et sexe obligatoires.

2.3) Gestion des pesées

Rôles concernés :

- Administrateur
- Employé

Règles de gestion :

- Chaque pesée est liée à un lot.
- Le gain de poids est calculé automatiquement.

Critères de validation :

- Lot obligatoire.
- Poids > 0.
- Date obligatoire.

2.4) Gestion de la reproduction

Rôles concernés :

- Administrateur
- Employé

Règles de gestion :

- Association d'un lot femelle et d'un lot mâle.
- Calcul automatique de la date prévue de mise bas.
- Respect de la cohérence des effectifs et des statuts.

Critères de validation :

- Date de saillie obligatoire.
- Femelles > 0.
- Mâles $\geq$ 1.
- Durée de gestation valide.

2.5) Analyse

Rôles concernés :

- Administrateur
- Employé

Règles de gestion :

- Calcul automatique des indicateurs.
- Génération automatique de la décision.
- Historisation des analyses.

Critères de validation :

- Lot existant.
- Taux entre 0 et 100 %.

2.6) Gestion sanitaire

Rôles concernés :

- Administrateur

- Employé

Règles de gestion :

- Historique des vaccinations.
- Suivi des rappels.

Critères de validation :

- Vaccin, lot et date obligatoires.

2.7) Gestion des stocks

Rôles concernés :

- Administrateur
- Employé

Règles de gestion :

- Types : ENTREE, SORTIE, AJUSTEMENT.
- Sortie autorisée uniquement si le stock est suffisant.
- Mise à jour automatique du stock.

Critères de validation :

- Ingrédient obligatoire.
- Quantité > 0.
- Date obligatoire.

2.8) Gestion commerciale et financière

Rôles concernés :

- Administrateur

Règles de gestion :

- Chaque vente est liée à un client.
- Les dépenses sont catégorisées.

Critères de validation :

- Client obligatoire.
- Montant > 0.

2.9) Tableau de bord

Rôles concernés :

- Administrateur
- Employé

Règles de gestion :

- Calcul automatique des indicateurs.
- Alertes selon les seuils.

Critères de validation :

- Données cohérentes.

2.10) Import / Export

Rôles concernés :

- Administrateur

Règles de gestion :

- Import des formats autorisés.
- Validation avant import.

Critères de validation :

- Fichier valide et non vide.

PARTIE 2: CHOIX TECHNIQUE ET ARCHITECTURE

1) Introduction

Pour ce projet, la stack retenue est **Java + Spring Boot** côté backend et **PostgreSQL** pour la base de données. Ce choix n'est pas arbitraire : c'est aujourd'hui l'une des combinaisons les plus solides, les plus éprouvées et les plus utilisées en entreprise pour construire une application robuste et durable. Voici pourquoi, argument par argument, avec les comparaisons face aux principales alternatives.

2) Choix technologique justifié

2.1) Backend

TECHNOLOGIE	ARGUMENTS
	• Le typage statique élimine les erreurs à la source. Le compilateur bloque tout code incohérent avant même l'exécution, alors qu'un langage dynamique comme Python ou JavaScript peut laisser passer ces erreurs jusqu'en production. • La JVM garantit une performance stable sous charge. Ce n'est pas le langage le plus rapide à écrire, mais c'est l'un des plus fiables une fois en exécution, grâce à des décennies d'optimisation. • Java est le standard des systèmes critiques (banque, assurance, grands SI), ce qui n'est pas un hasard : c'est un choix de fiabilité, pas de mode.
	• L'injection de dépendances est le vrai différenciateur. Elle permet de découpler le code et de tester chaque composant isolément. Express.js ne l'offre pas nativement : il faut tout gérer à la main ou ajouter des librairies tierces non standardisées. • L'écosystème Spring est unifié, alors que Node.js oblige souvent à choisir entre plusieurs librairies concurrentes de qualité inégale pour une même fonctionnalité (authentification, ORM, validation...). • Spring Boot est "production-ready" dès le départ : monitoring (Actuator), sécurité (Spring Security), accès aux données (Spring Data JPA) sont pensés pour fonctionner ensemble sans configuration lourde.

2.2) Base de données

TECHNOLOGIE	ARGUMENTS
	• PostgreSQL garantit l'intégrité des données (ACID strict), un point non négociable dès qu'on manipule des données utilisateurs ou des transactions. • PostgreSQL combine relationnel et flexibilité grâce au format JSONB : il n'est pas nécessaire de sacrifier la rigueur d'une base relationnelle pour gagner en flexibilité, contrairement à ce qu'impose souvent MongoDB. • PostgreSQL est gratuit et sans limite de licence, contrairement à Oracle ou SQL Server qui impliquent des coûts significatifs, sans apporter d'avantage décisif pour ce type de projet.

2.3) Moteur de templates

TECHNOLOGIE	ARGUMENTS
	• Une technologie simple et bien maîtrisée : JSP est une technologie simple à prendre en main . Elle est "basée sur HTML", ce qui la rend accessible. Dans une architecture MVC, JSP joue parfaitement son rôle de couche de présentation, en se concentrant sur l'affichage des données que les contrôleurs lui envoient. • Cohérence dans l'écosystème Java EE : JSP est une technologie standard de l'écosystème Java/Jakarta EE, dans lequel Spring Boot s'inscrit. Pour des développeurs formés à cet environnement, c'est un environnement familier, ce qui facilite la maintenance .

3) Comparaison des technologies – Projet Madaporc

3.1) Backend : Spring Boot vs Laravel

Critère	Argument	Avantage
Gestion des modules métier	Architecture en couches adaptée aux modules Lots, Reproduction, Analyse et Stock.	Spring Boot
Calculs métier	Centralisation des calculs (fertilité, aptitude, recommandations, stock) dans les Services.	Spring Boot
Relations de données	Spring Data JPA simplifie les relations entre les entités.	Spring Boot
Évolutivité	Ajout de nouveaux modules sans impacter les existants.	Spring Boot
Déploiement	Application JAR autonome avec Tomcat embarqué.	Spring Boot

3.2) Java 17 vs PHP

Critère	Argument	Avantage
Fiabilité des calculs	Utilisation de BigDecimal pour les taux et indicateurs.	Java 17
Typage	Enums et LocalDate réduisent les erreurs métier.	Java 17
Organisation	Séparation DTO, Services, Repositories et Models.	Java 17
Maintenance	Les règles métier évoluent sans modifier les vues.	Java 17

3.3) PostgreSQL vs Oracle Database

Critère	Argument	Avantage
Contraintes métier	Support des CHECK, FOREIGN KEY, UNIQUE et colonnes calculées.	Égalité
Volume de données	Adapté aux besoins d'un élevage.	PostgreSQL
Coût	Gratuit et sans licence.	PostgreSQL
Intégration	Excellente intégration avec Spring Data JPA.	Égalité

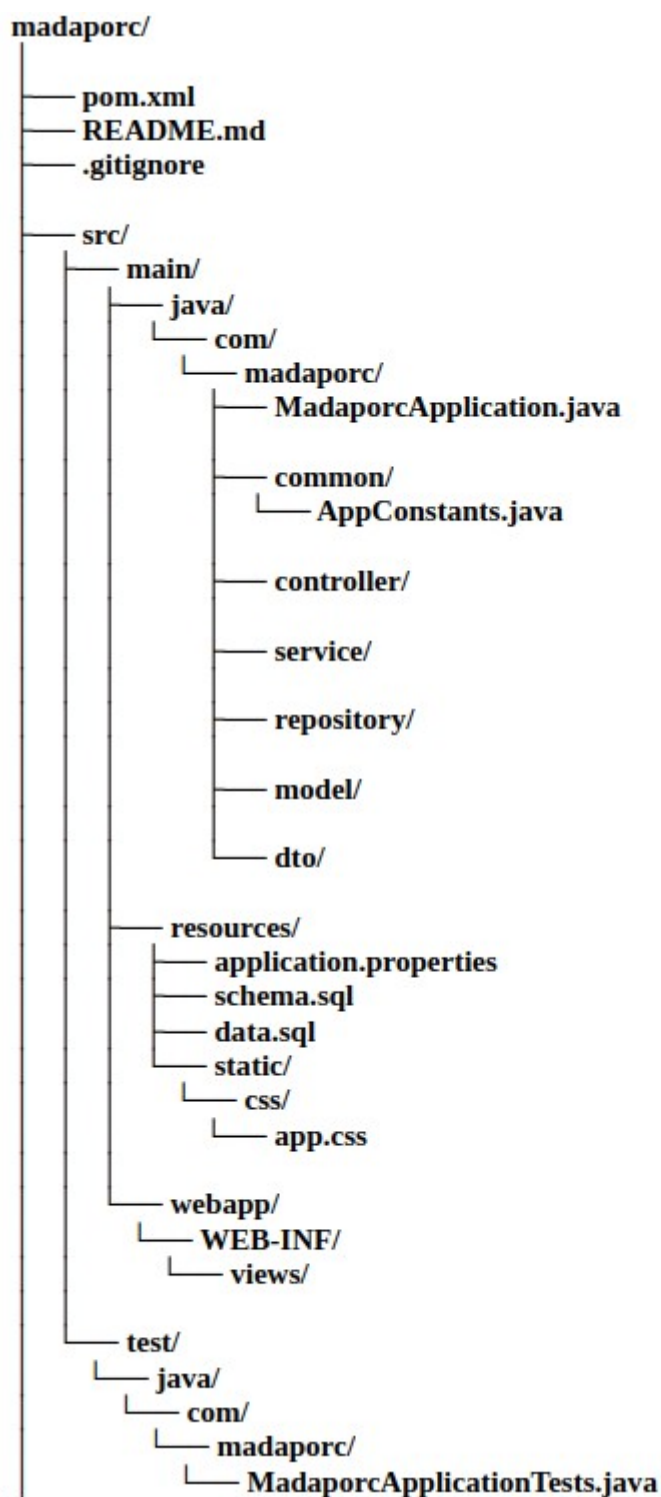
3.4) JSP vs Thymeleaf

Critère	Argument	Avantage
Compatibilité	Les interfaces sont déjà développées en JSP.	JSP
Affichage	JSTL et EL couvrent les besoins du projet.	Égalité
Maintenance	Correcte si la logique reste côté Service.	JSP
Évolution	Thymeleaf est recommandé pour les nouveaux projets.	Thymeleaf

4) Architecture globale du projet:

Le
une

projet a
recours à

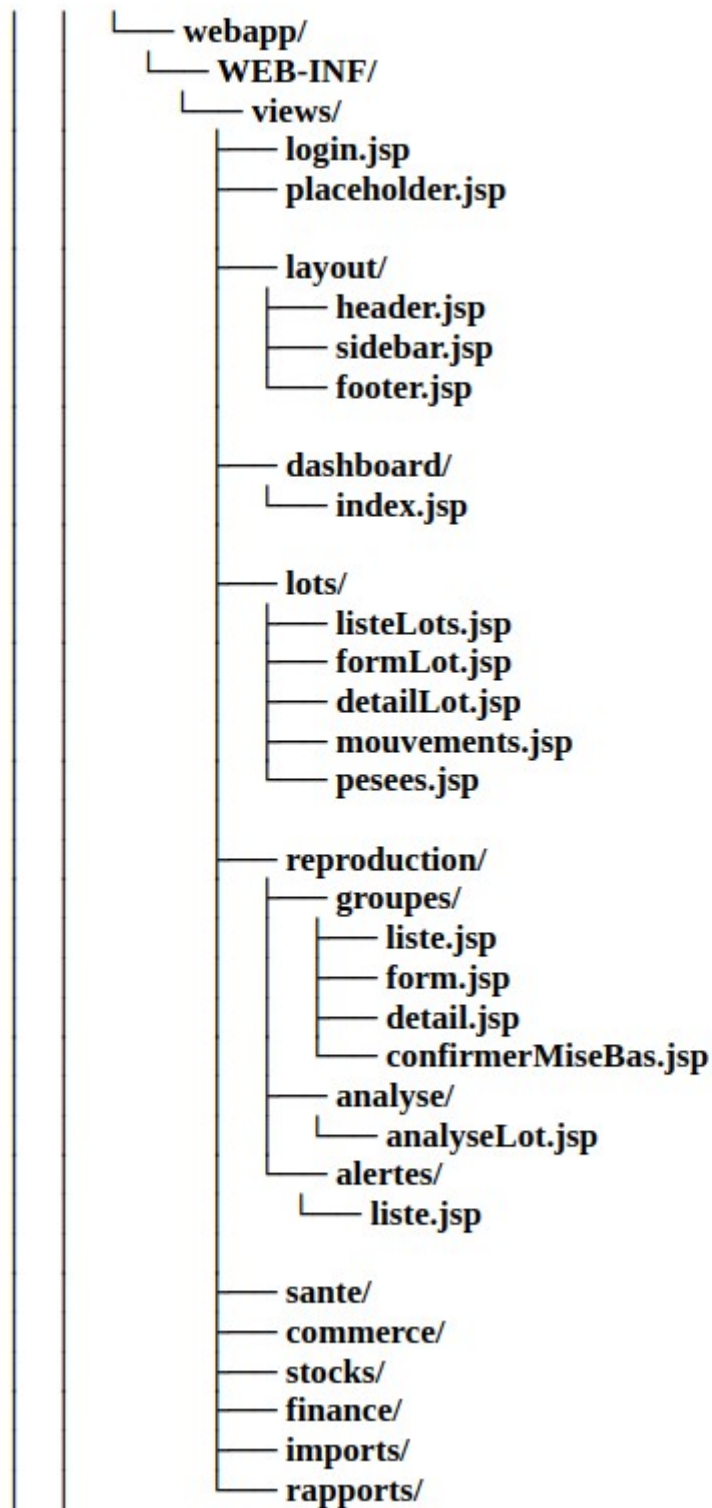


architecture MVC en couches. Chaque couche possède un rôle bien précis.

4.1) Couche de présentation (Views):

Il s'agit

de

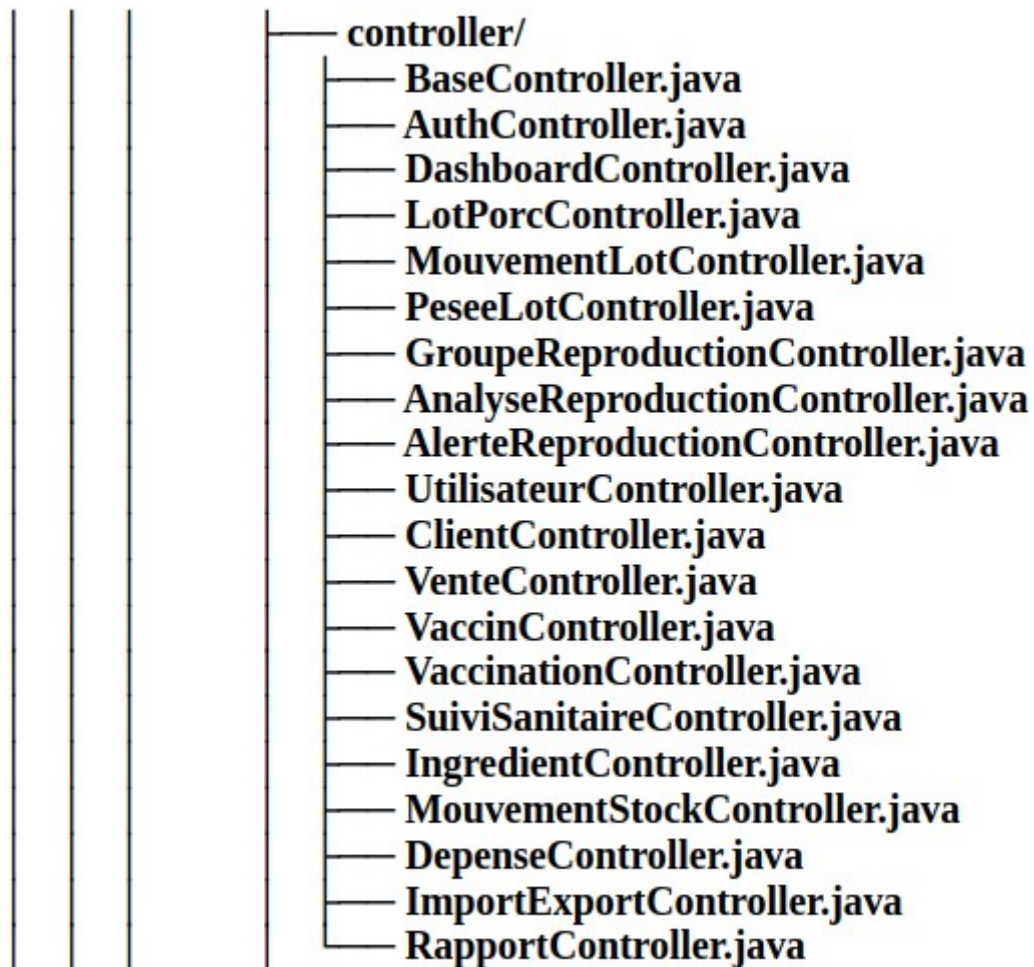


l'interface user, elle a pour rôle de:

- afficher les données
- récupère les saisies user
- envoie les formulaires

4.2) Couche Contrôleur (Controller):

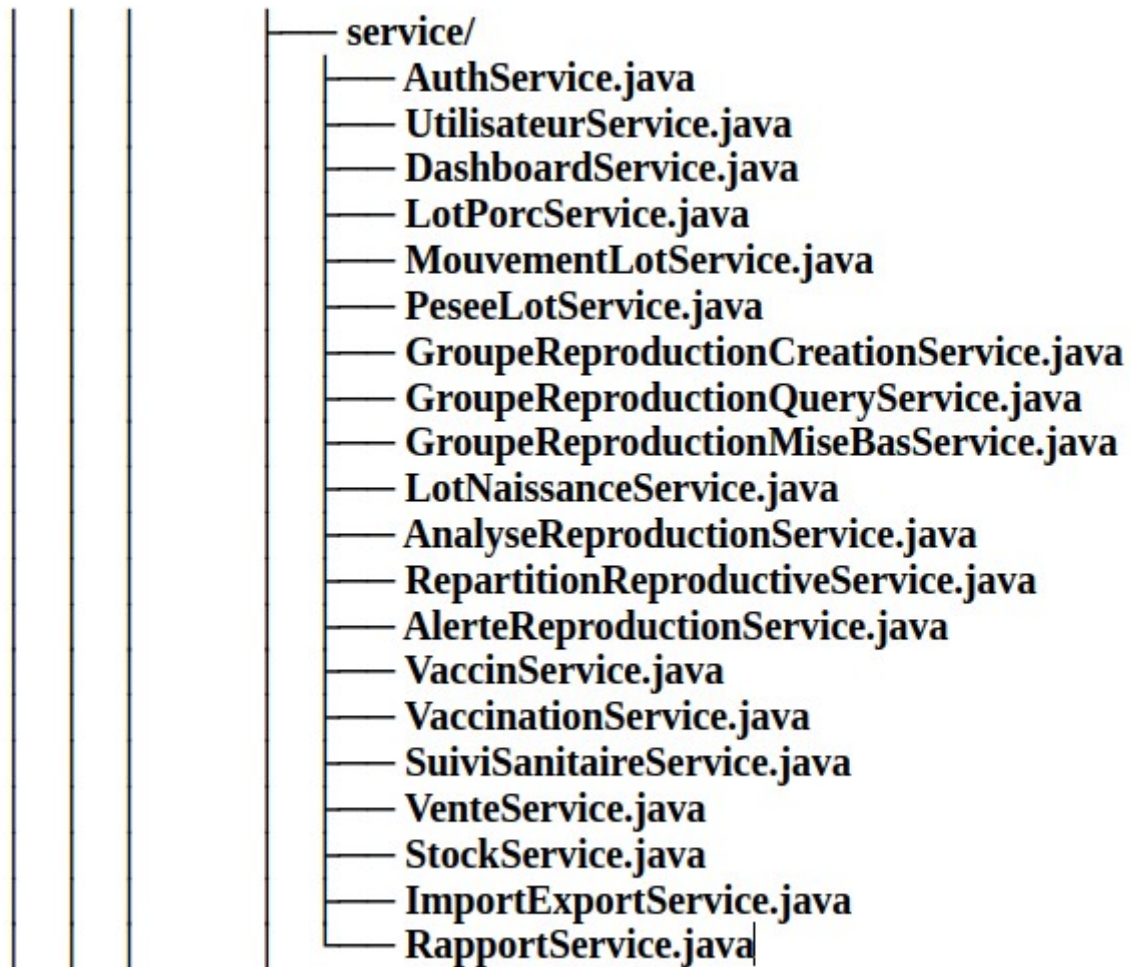
Les



contrôleurs:

- reçoivent les requêtes HTTP
- appellent les services
- transmettent les données vers les Views

4.3) Couche Service:



Cette couche :

- effectue les calculs
- applique les règles métier
- valide les données
- coordonne plusieurs repositories

4.4) Couche Model:

Les					└─ model/
					└─ Role.java
					└─ Utilisateur.java
					└─ Race.java
					└─ ParametreReproductionRace.java
					└─ LotPorc.java
					└─ MouvementLotPorc.java
					└─ PeseeLot.java
					└─ StatutReproductif.java
					└─ RepartitionReproductiveLot.java
					└─ GroupeReproduction.java
					└─ AnalyseReproductionLot.java
					└─ AlerteReproduction.java
					└─ Vaccin.java
					└─ Vaccination.java
					└─ Maladie.java
					└─ Traitement.java
					└─ SuiviSanitaire.java
					└─ Client.java
					└─ Vente.java
					└─ DetailVente.java
					└─ Ingredient.java
					└─ Depense.java
					└─ ImportExport.java

modèles représentent les tables. Chaque attribut Java correspond à une colonne SQL.

4.5) Couche DTO:



servent :

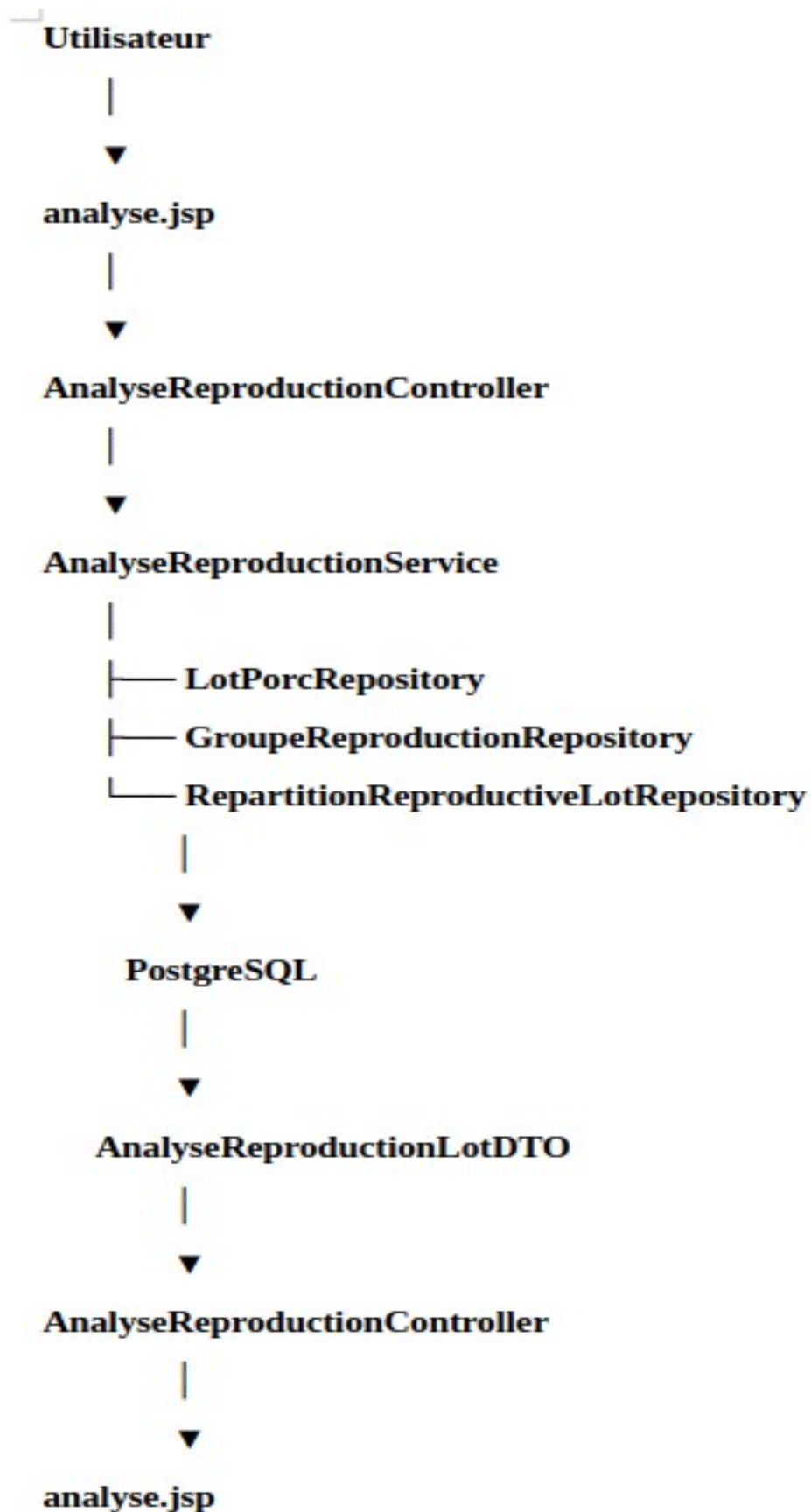
- à transporter des données
- éviter de transmettre directement les entités.

5) WorkFlow du projet:

Prenons comme exemple l'écran Analyse de reproduction:

Déroulement de l'analyse:

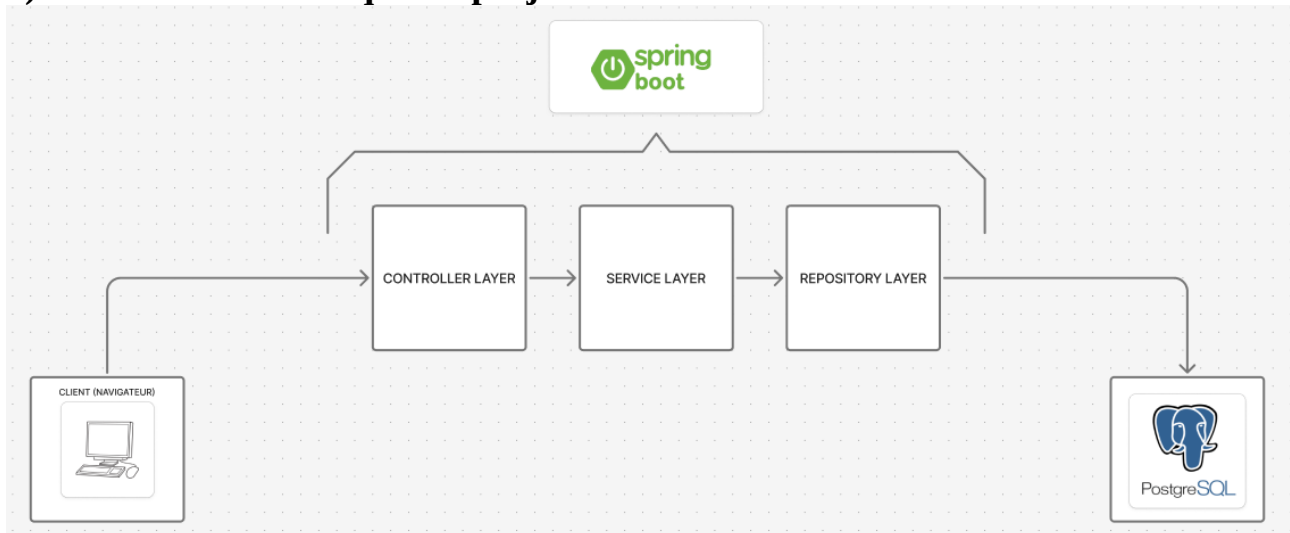
1. L'utilisateur clique sur un lien "Voir analyse" dans son navigateur.
2. Le navigateur envoie une requête HTTP GET vers l'URL correspondante.
3. Spring MVC route la requête vers le contrôleur dédié
(ex: `ReproductionController`).
4. Le contrôleur appelle le service d'analyse (`ReproductionService`)
pour obtenir les indicateurs calculés.
5. Le service effectue les calculs métier, interroge les repositories
(`LotRepository`, `ReproductionRepository`, etc.) si nécessaire.
6. Les repositories exécutent des requêtes SQL via JPA/Hibernate
pour récupérer les données de la base PostgreSQL.
7. Les données remontent : `Repository` → `Service` → `Contrôleur`.
8. Le contrôleur place les données dans le modèle (`Model`) et
renvoie le nom de la vue JSP (ex: "analyse-reproduction").
9. Spring MVC charge la vue JSP correspondante, qui utilise EL et JSTL
pour afficher dynamiquement les données.
10. La page HTML est renvoyée au navigateur de l'utilisateur.



Cette architecture présente plusieurs avantages :

- **Séparation des responsabilités** : chaque couche a un rôle clair.
- **Maintenance facilitée** : une modification dans une couche impacte peu les autres.
- **Réutilisabilité** : les services peuvent être appelés par plusieurs contrôleurs.
- **Testabilité** : il est plus simple de tester les services indépendamment des vues et de la base de données.

6) Architecture technique du projet:



◆ Autres technologies utilitaires:



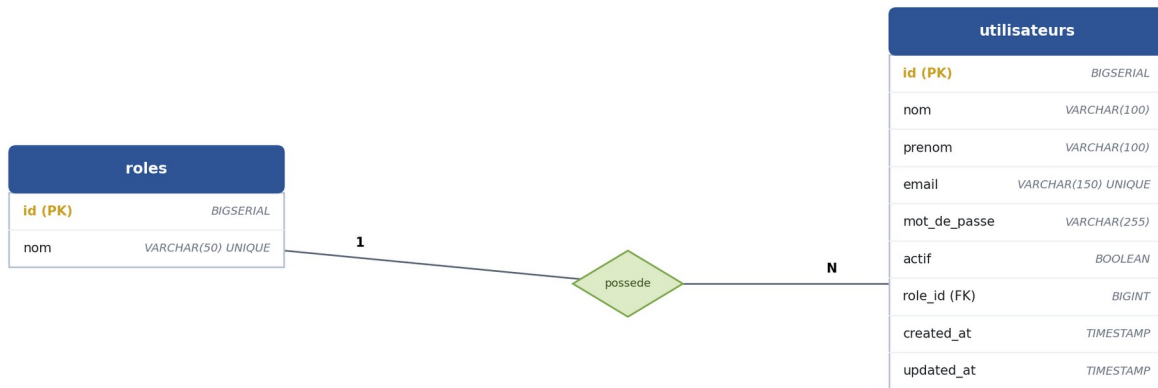
JavaScript



CSS

PARTIE 3: CONCEPTION DE LA BASE DE DONNÉES

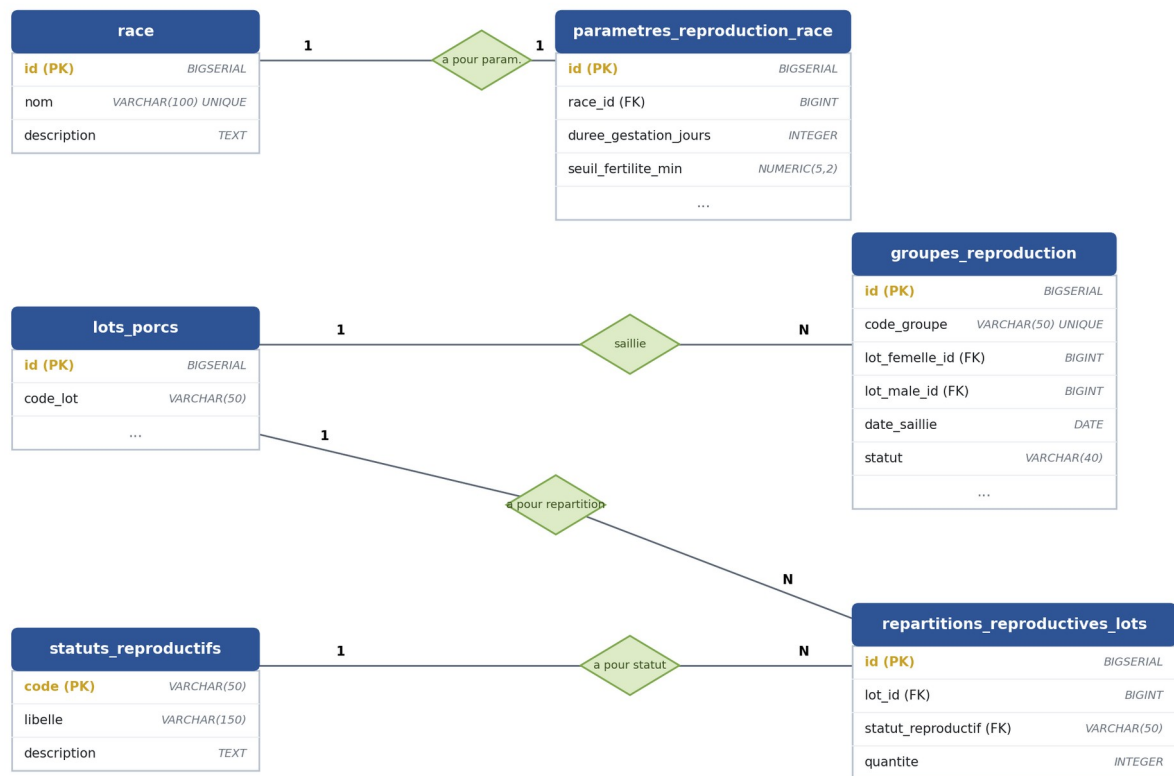
1) Module Gestion des users:



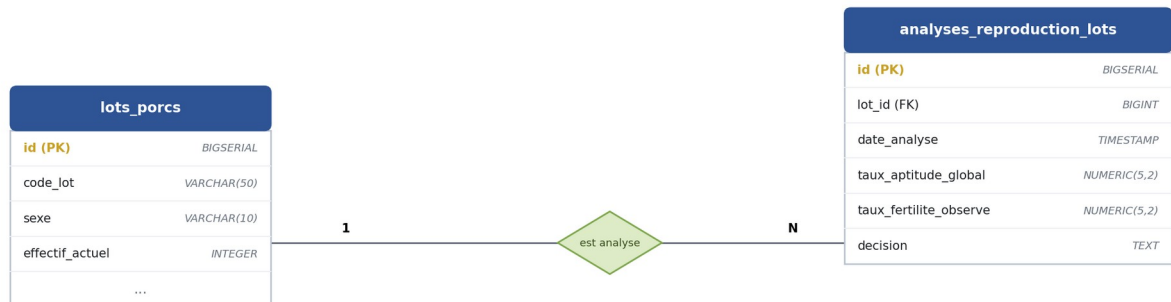
2) Module Gestion des porcs:



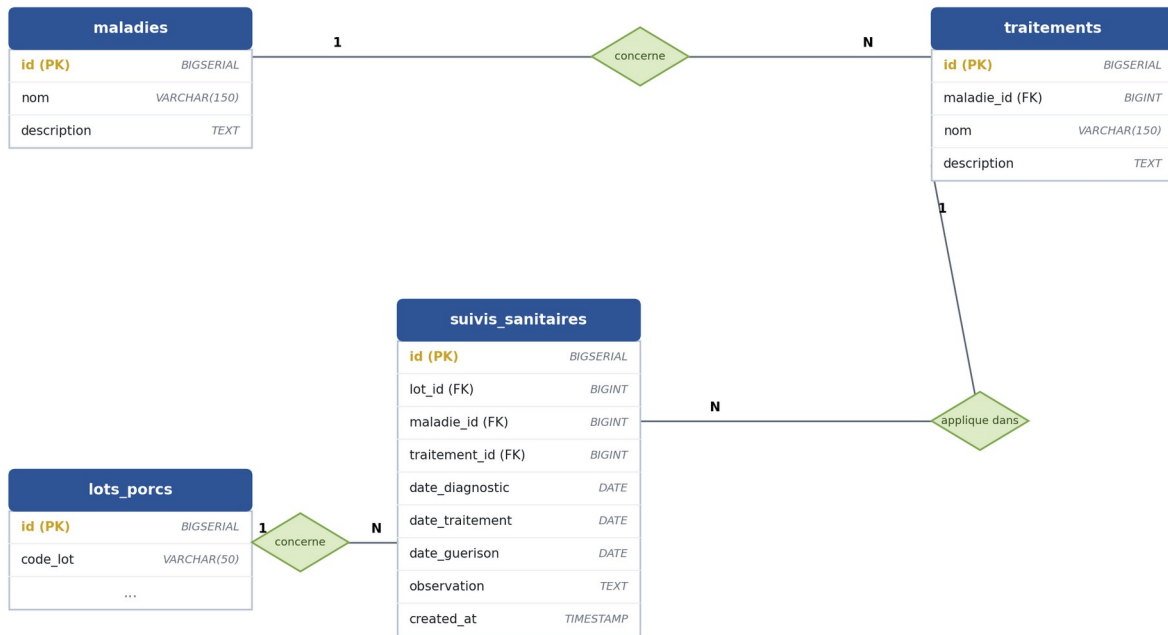
3) Module reproduction:



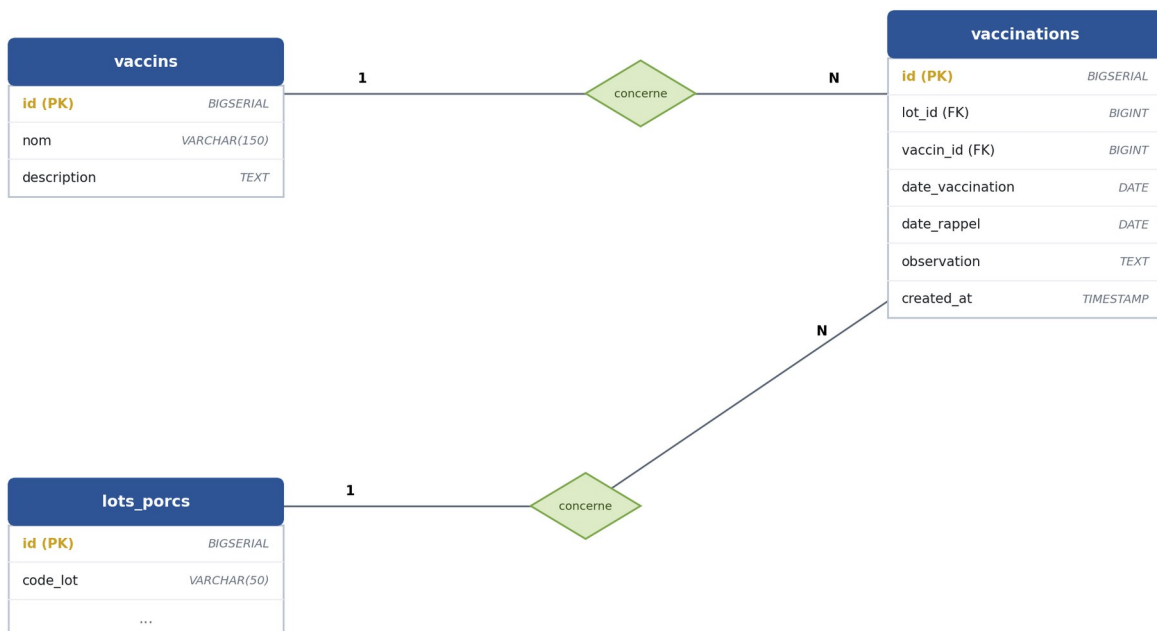
4) Module Analyse de reproduction:



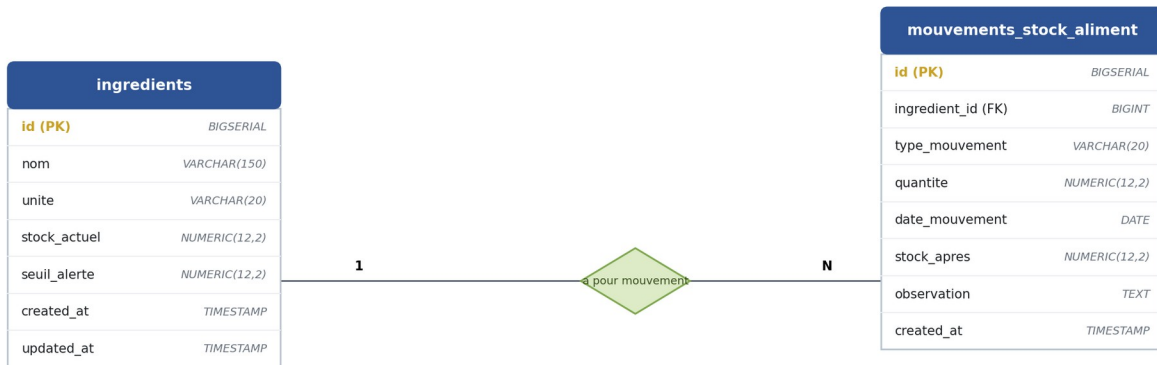
5) Module sanitaire:



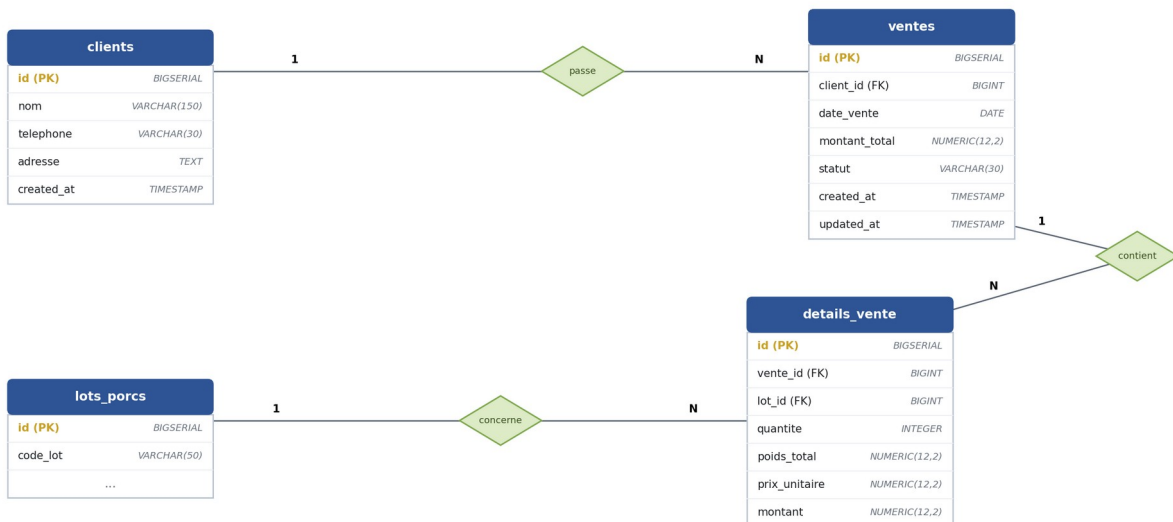
6) Module vaccinations:



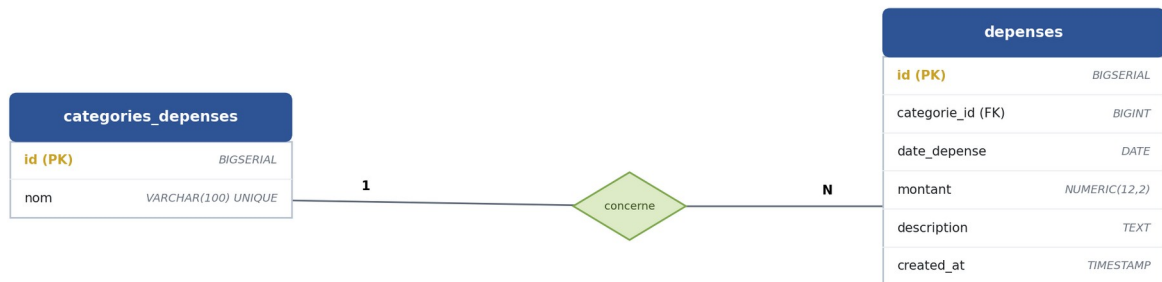
7) Module Stock Alimentaire:



8) Module commercial (clients / ventes):



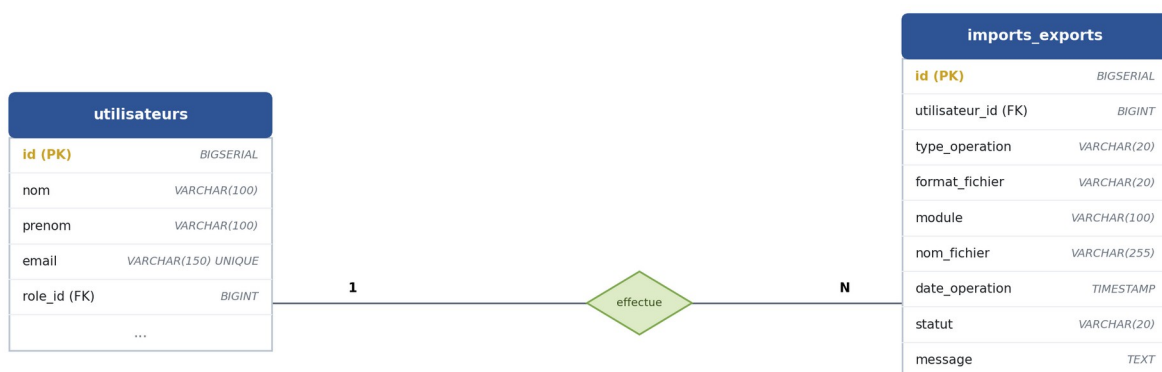
9) Module dépenses:



10) Module Alertes:



11) Module Import-Export:

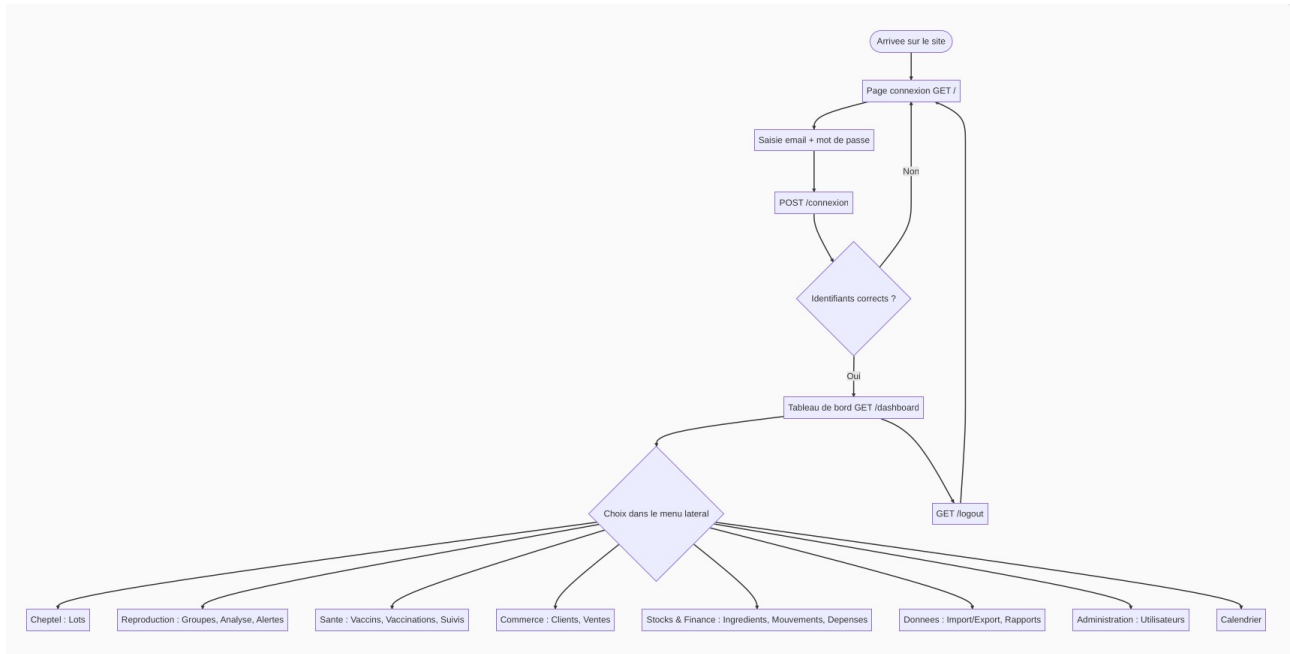


PARTIE 4: PILOTAGE DU PROJET ET PERSPECTIVES

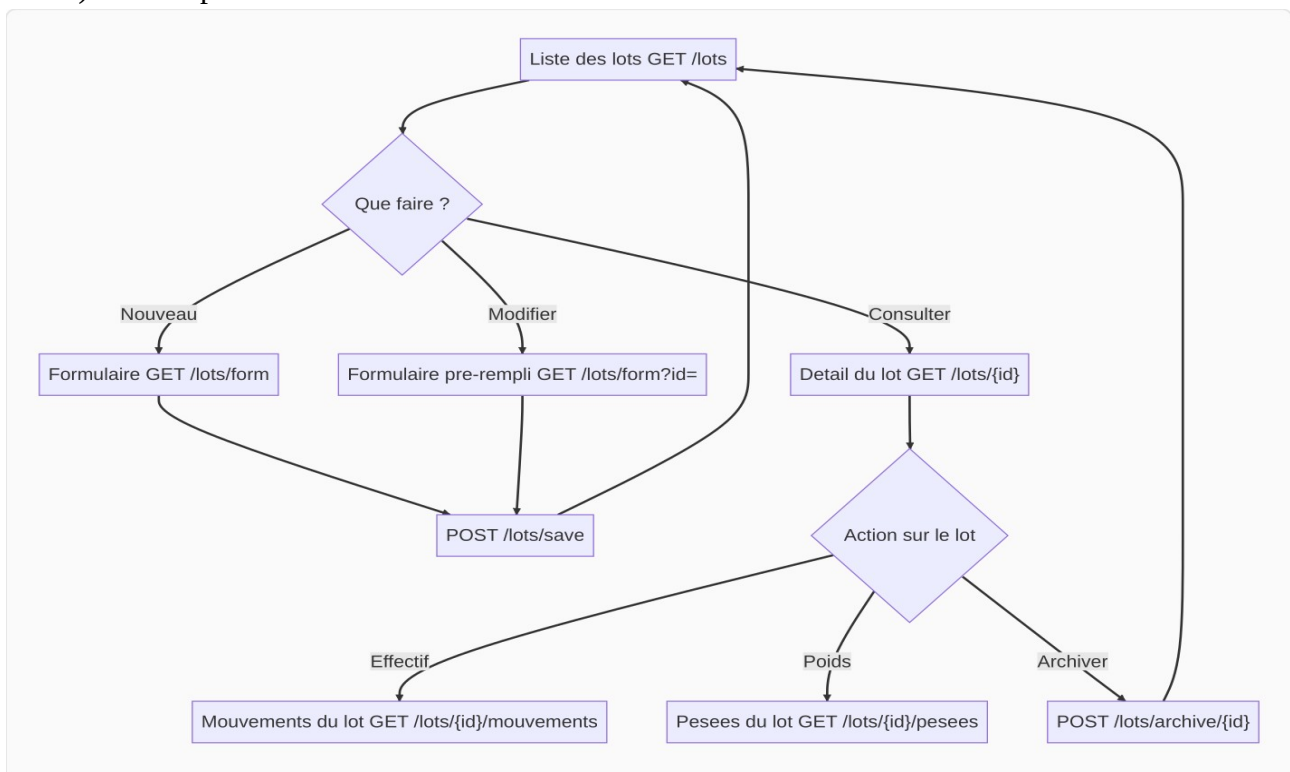
1) Organisation du projet:

1.1) Userflow:

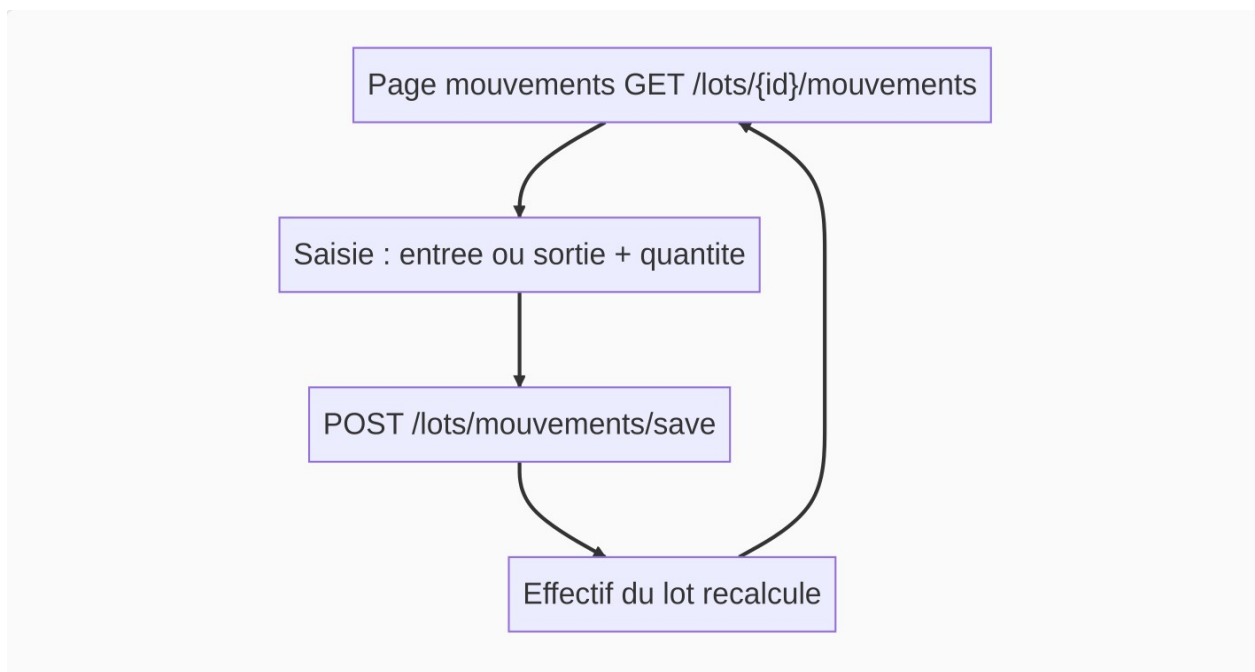
1.1.1) Connexion et navigation générale



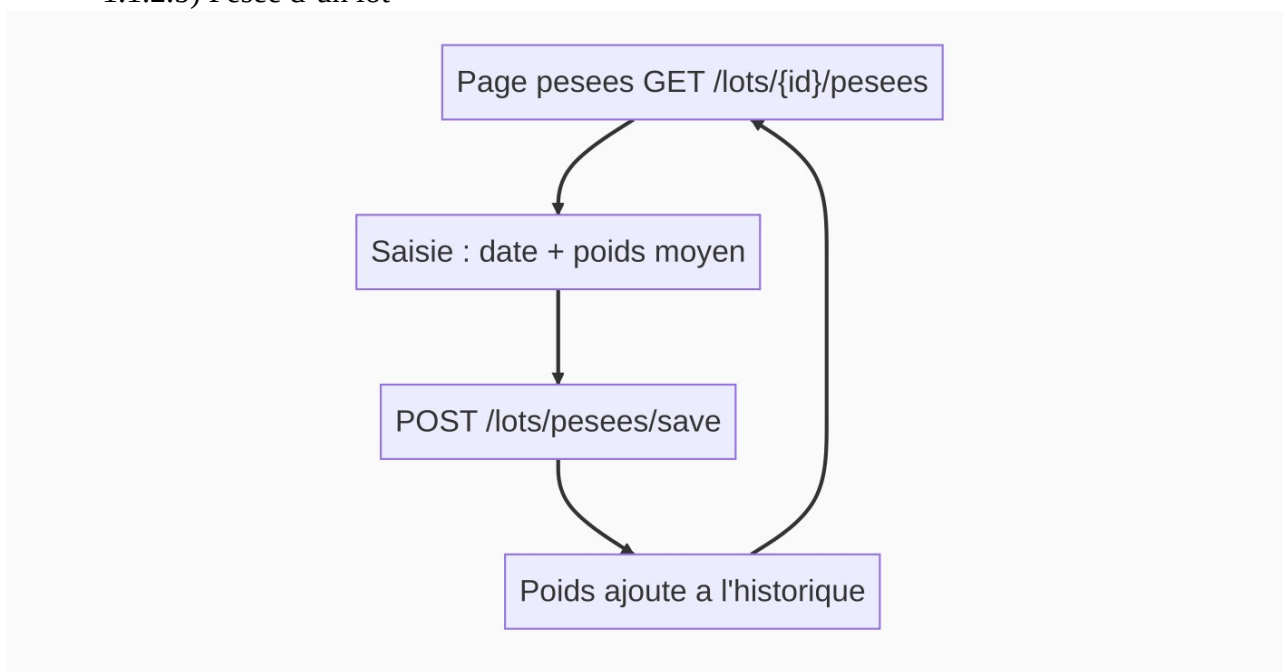
1.1.2) Lots de porcs



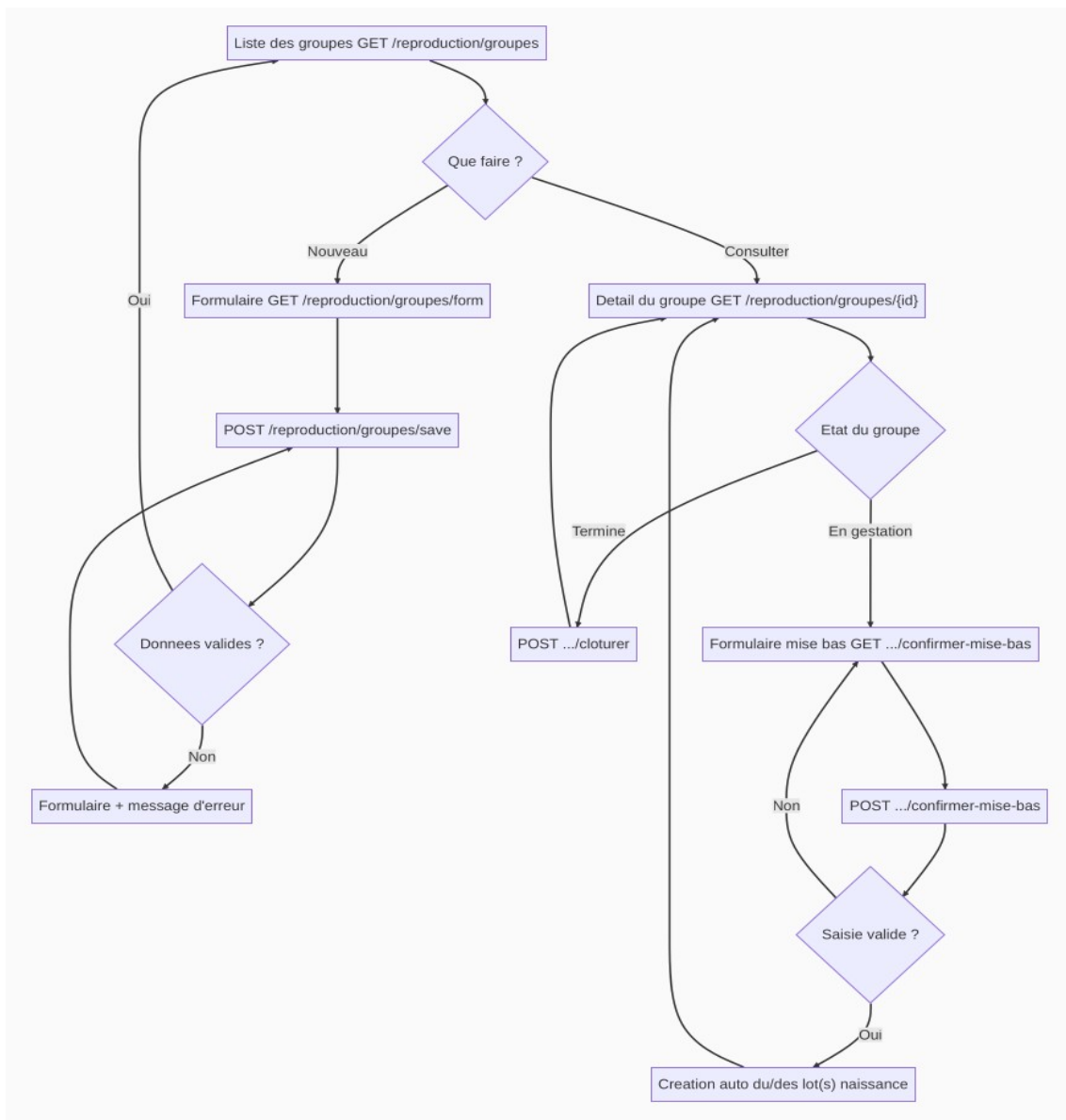
1.1.2.a) Mouvement d'effectif d'un lot



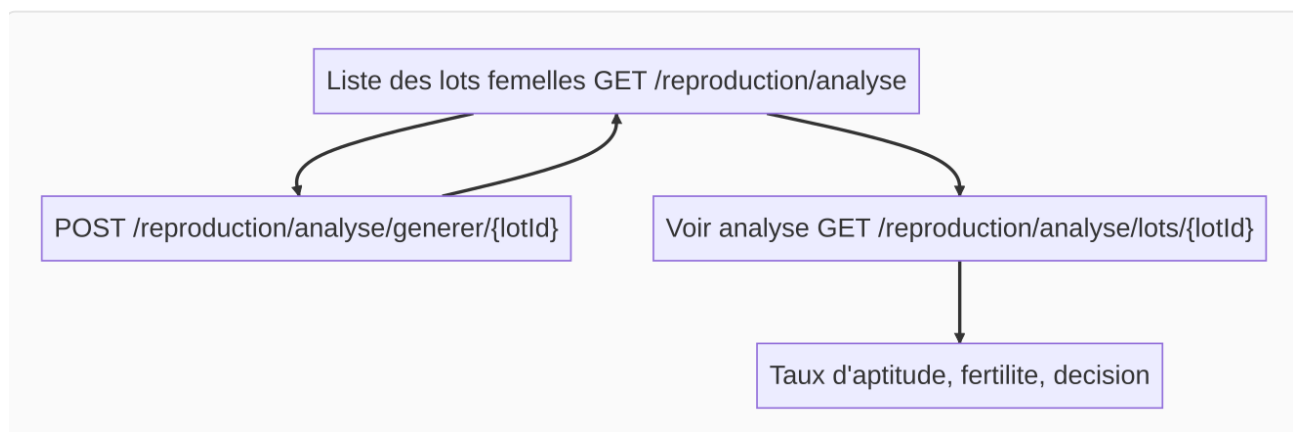
1.1.2.b) Pesée d'un lot



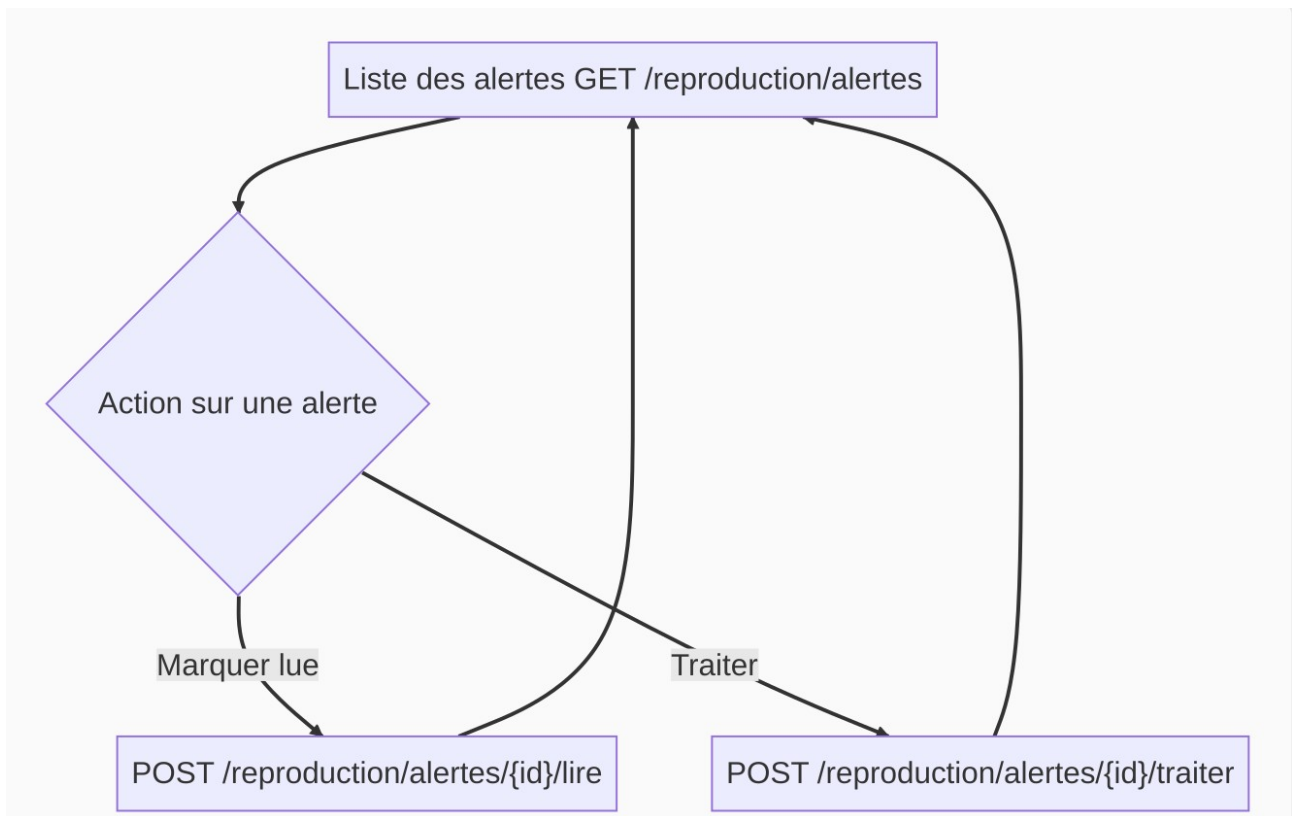
1.1.3) Groupe de reproduction



1.1.4) Analyse reproductive

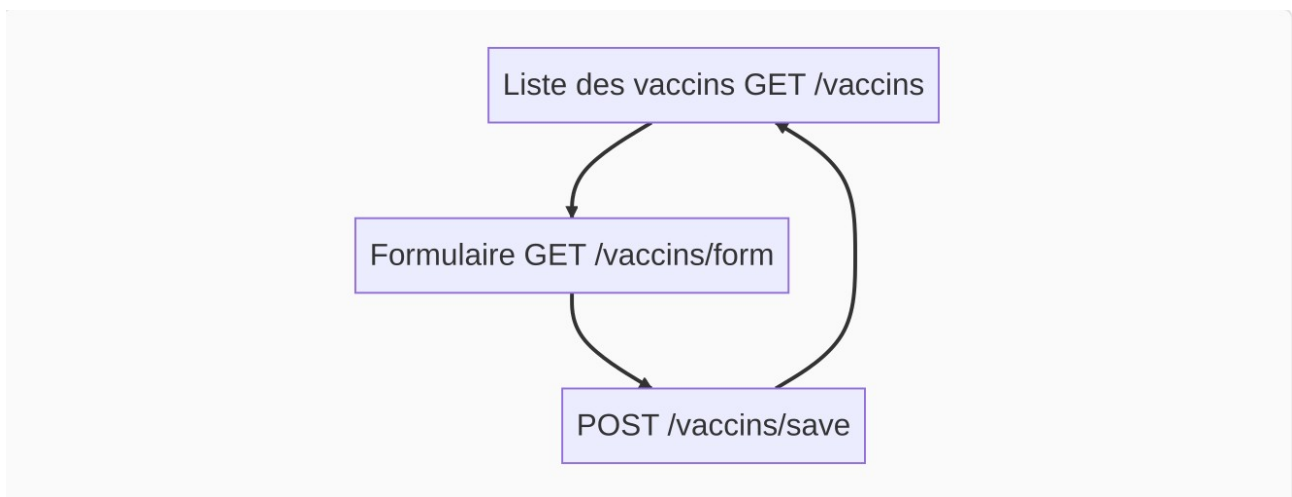


1.1.5) Alerte de reproduction

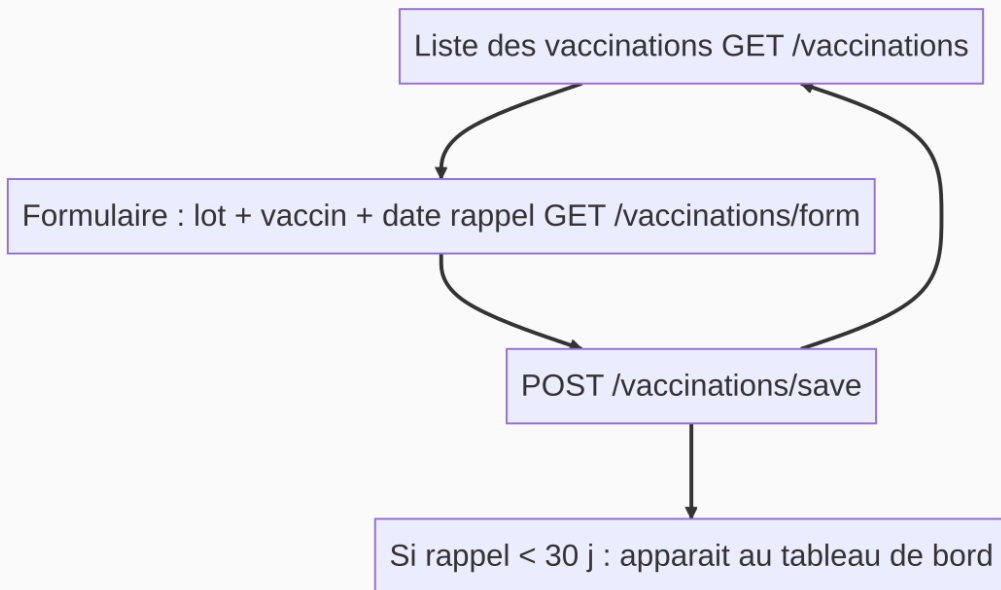


1.1.5) Sante

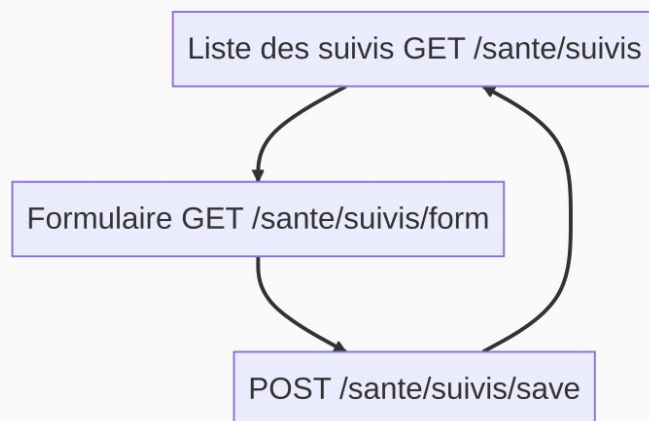
1.1.5.a) Vaccin



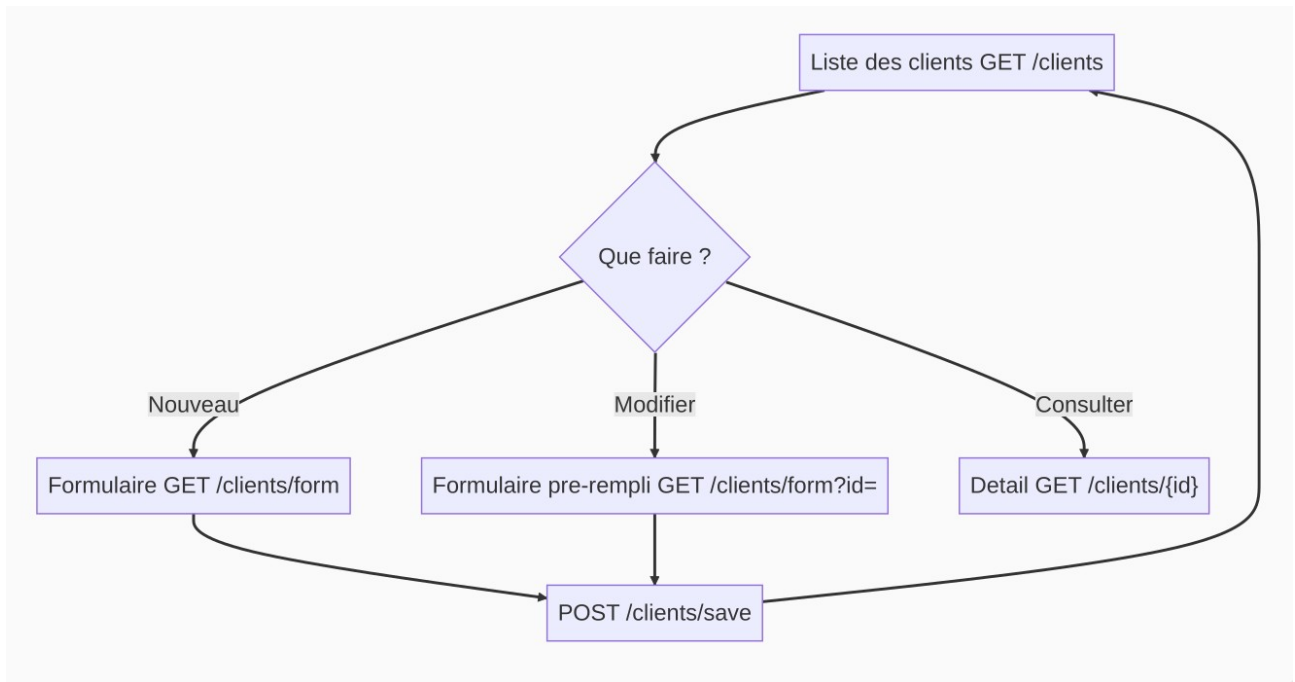
1.1.5.b) Vaccination



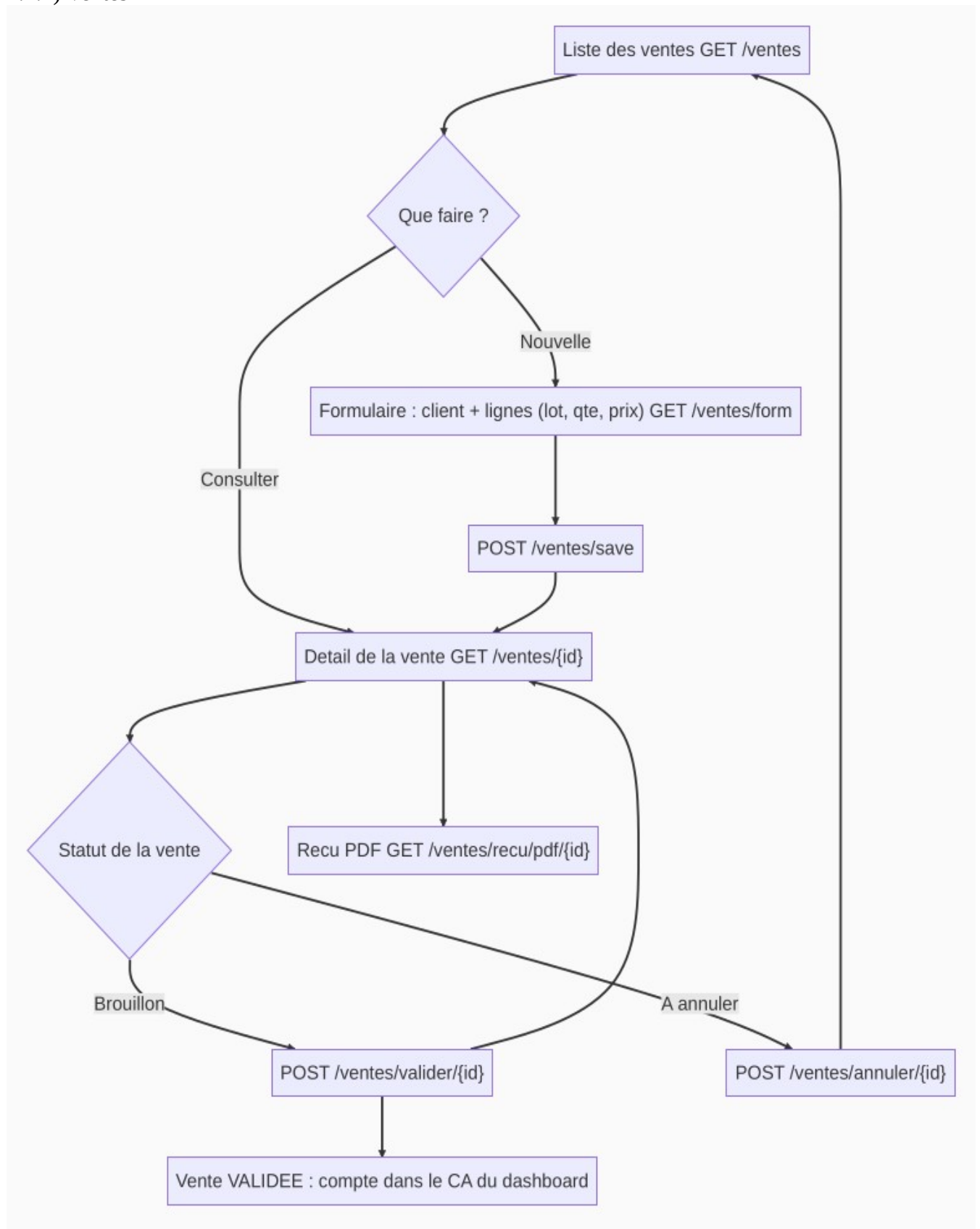
1.1.5.c) Suivre sanitaire



1.1.6) Clients

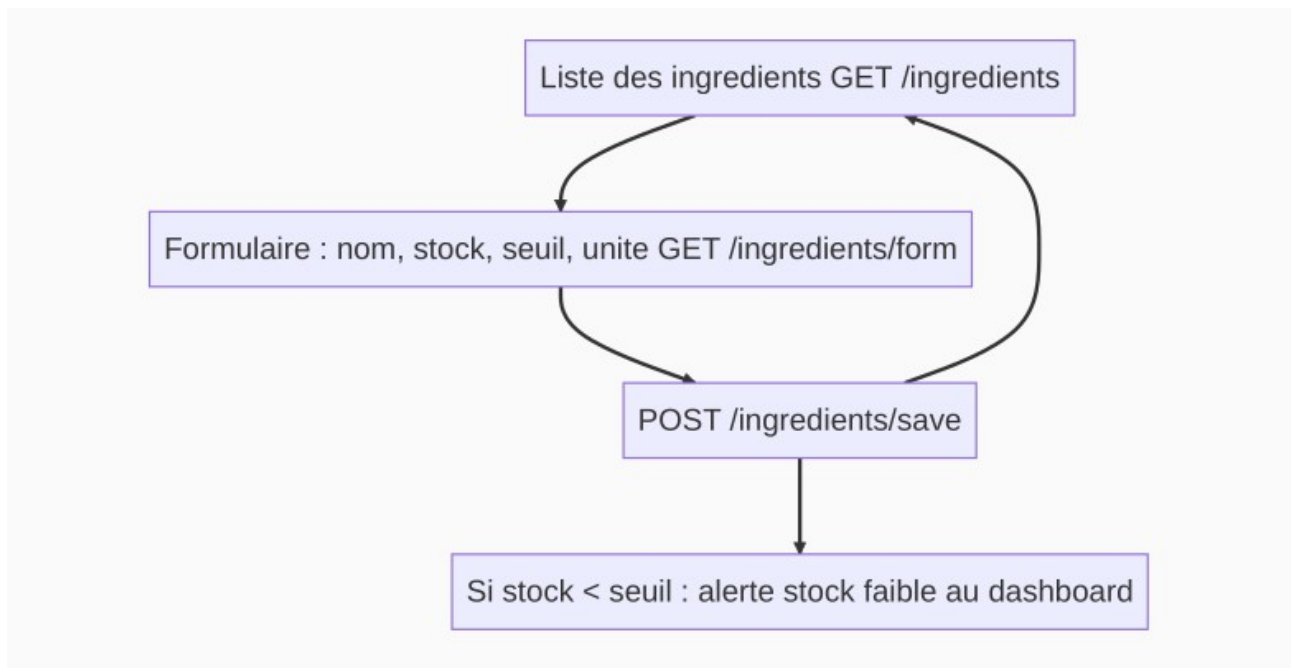


1.1.7) Ventes

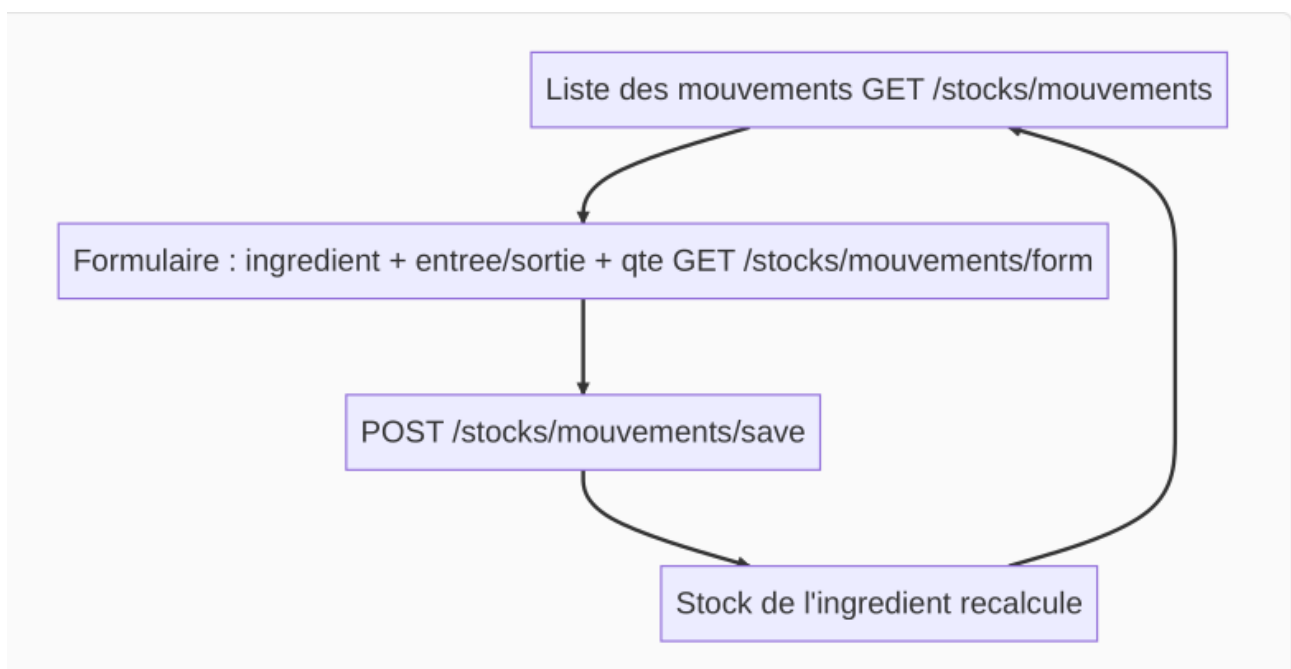


1.1.8) Stocks et finance

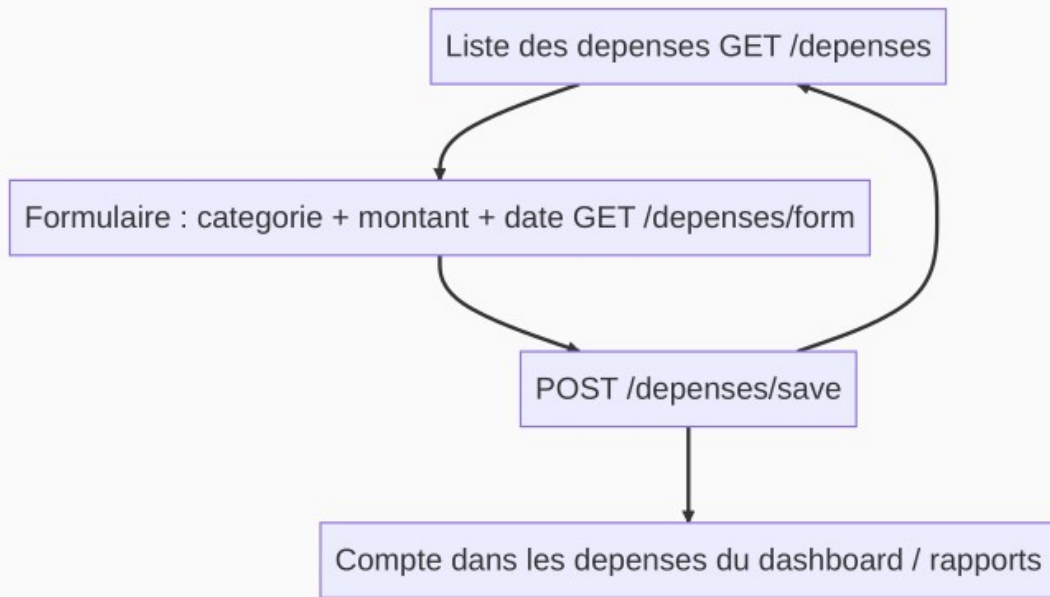
1.1.8.a) Ingredients



1.1.8.b) Mouvements de stocks

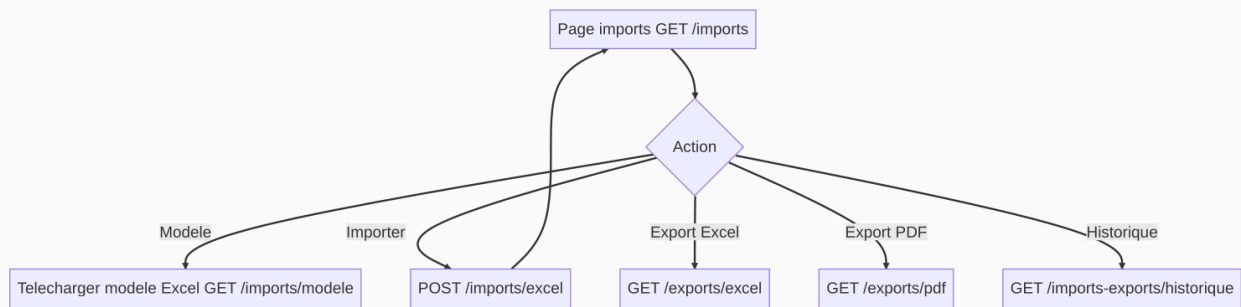


1.1.8.c) Dépense

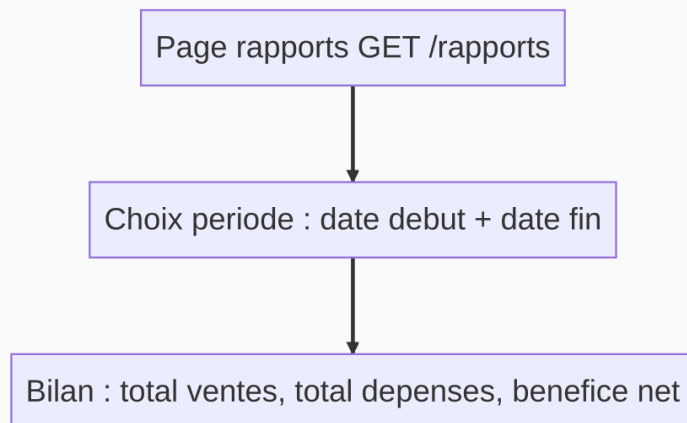


1.1.9) Donnees

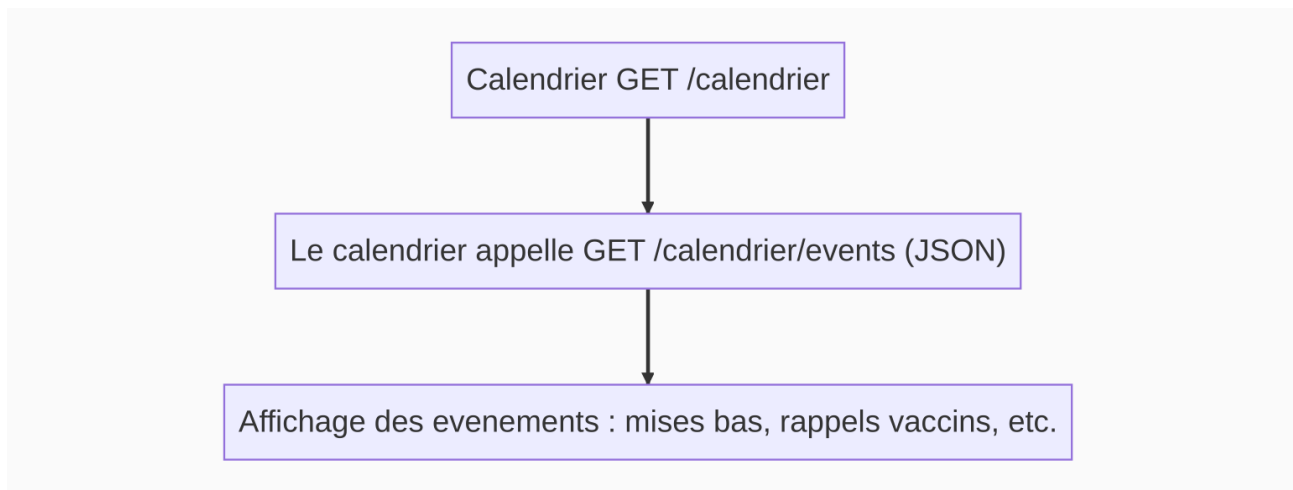
1.1.9.a) Import / Export



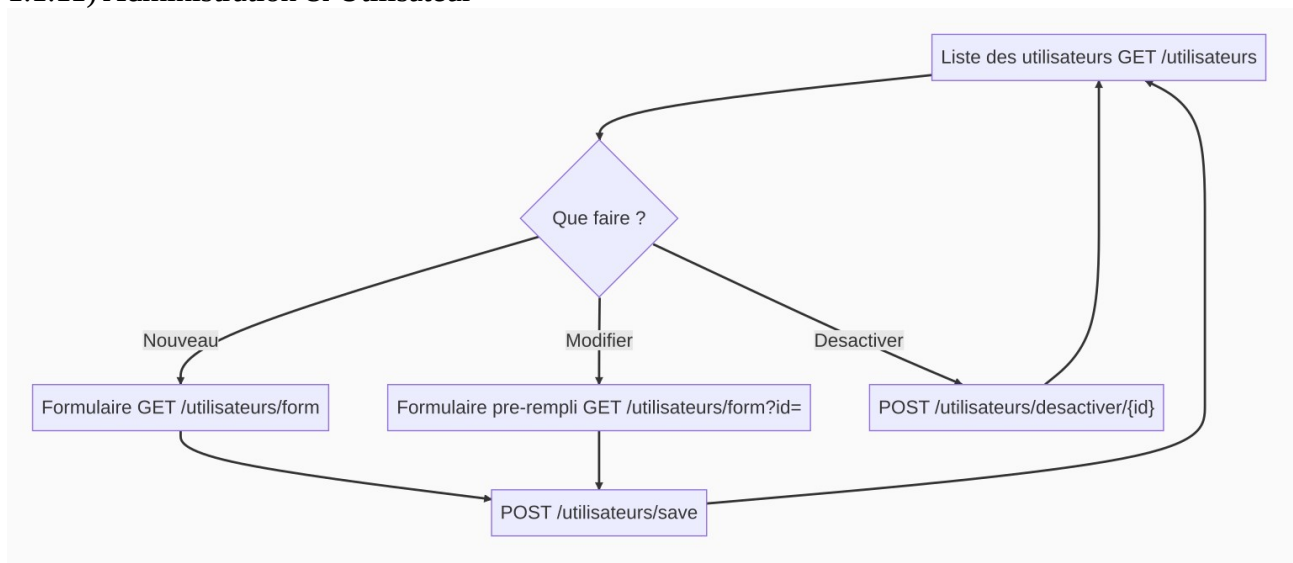
1.1.9.b) Rapport



1.1.10) Calendrier



1.1.11) Administration & Utilisateur



1.2) Planning:

1.2.a) Gant

1.2.a) Excel

Mickaella (chef de projet)

N°	Partie	Couche	Fichier	Fonction / Route (signature)	Durée (min)	Statut	Remarque
1	1ère part	Controller	controller/	@GetMapping(/	30	Terminé	

			GroupeReproductionC ontroller.java	reproduction/groupes/{id}) detail(id, Model)			
2	1ère part	Controller	controller/ GroupeReproductionC ontroller.java	@GetMapping("/{id}/ confirmer-mise-bas) afficherFormulaireMiseBas(i d, Model)	30	Terminé	#N/A
3	1ère part	Controller	controller/ GroupeReproductionC ontroller.java	@PostMapping("/{id}/ confirmer-mise-bas) confirmerMiseBas(id, dto, Model)	30	Terminé	
4	1ère part	Controller	controller/ GroupeReproductionC ontroller.java	@PostMapping("/{id}/ cloturer) cloturer(id)	30	Terminé	Implémentée
5	1ère part	Service	service/ GroupeReproduction MiseBasService.java	getDetailGroupe(groupeId)	30	Terminé	
6	1ère part	Service	service/ GroupeReproduction MiseBasService.java	calculerJoursRestants(groupe Id)	30	Terminé	
7	1ère part	Service	service/ GroupeReproduction MiseBasService.java	calculerPourcentageEvolutio n(groupeId)	30	Terminé	
8	1ère part	Service	service/ GroupeReproduction MiseBasService.java	confirmerMiseBas(groupeId, dto)	30	Terminé	
9	1ère part	Service	service/ GroupeReproduction MiseBasService.java	validerDonneesMiseBas(dto)	30	Terminé	Présente
10	1ère part	Service	service/ GroupeReproduction MiseBasService.java	mettreAJourGroupeApresMi seBas(groupeId, dto)	30	Terminé	Présente
11	1ère part	Service	service/ GroupeReproduction MiseBasService.java	mettreAJourRepartitionApre sMiseBas(lotId, groupe)	30	Terminé	Via repartitionReprodu ctiveService.mettre AJourApresMiseBa s
12	1ère part	Service	service/ GroupeReproduction MiseBasService.java	cloturer(groupeId)	30	Terminé	Implémentée
13	1ère part	Service	service/ LotNaissanceService.j ava	creerLotNaissanceApresMis eBas(groupeId, dto)	30	Terminé	Implémentée (création lot + lien lot mère/groupe)
14	1ère part	Service	service/ LotNaissanceService.j ava	preparerLotNaissanceDepuis Groupe(groupeId)	30	Terminé	Implémentée
15	1ère part	DTO	dto/ ConfirmationMiseBas DTO.java · LotNaissanceDTO.jav a	DTO mise bas + lot naissance	20	Terminé	
16	1ère part	Vue	reproduction/ groupes/detail.jsp · confirmerMiseBas.jsp	Détail groupe + barre évolution + confirmation	40	Terminé	
17	Suite	Controller	controller/ RapportController.jav a	@GetMapping(/rapports) index(RapportFiltreDTO, Model)	30	Terminé	Implémenté (bilan financier filtré par dates)
18	Suite	Service	service/ RapportService.java	genererRapport · calculerTotalVentes (VALIDEE) · calculerTotalDepenses · calculerBeneficeNet · compterVentes · compterDepenses · validerFiltre	210	Terminé	Implémenté
19	Suite	DTO	dto/ RapportFiltreDTO.jav a	Filtre dates rapport (dateDebut, dateFin)	10	Terminé	Complété
20	Suite	DTO	dto/ RapportFinancierDTO .java	totalVentes · totalDepenses · beneficeNet · nbVentes · nbDepenses	10	Terminé	Créé
21	Suite	Vue	rapports/index.jsp	Bilan financier + filtre dates	20	Terminé	Bloc financier ajouté
TOTAL (min)					760		

Tsanta (chef Interime)

N°	Partie	Couche	Fichier	Fonction / Route (signature)	Durée (min)	Statut	Remarque
1	1ère part	Controller	controller/ GroupeReproductionC ontroller.java	@GetMapping(/ reproduction/groupes) listGroupes(Model)	30	Terminé	
2	1ère part	Controller	controller/ GroupeReproductionC ontroller.java	@GetMapping(/ reproduction/groupes/form) showForm(id, Model)	30	Terminé	
3	1ère part	Controller	controller/ GroupeReproductionC ontroller.java	@PostMapping(/ reproduction/groupes/save) save(dto, Model,	30	Terminé	

				HttpSession)			
4	1ère part	Service	service/ GroupeReproductionC reationService.java	creer(dto, utilisateurId)	30	Terminé	
5	1ère part	Service	service/ GroupeReproductionC reationService.java	modifier(id, dto)	30	Terminé	
6	1ère part	Service	service/ GroupeReproductionC reationService.java	calculerDatePrevueMiseBas(dateSaillie, dureeGestation)	30	Terminé	
7	1ère part	Service	service/ GroupeReproductionC reationService.java	verifierLotFemelle · verifierLotMale · verifierFemellesDisponibles	90	Terminé	
8	1ère part	Service	service/ GroupeReproductionC reationService.java	mettreAJourRepartitionApre sSaillie(lotId, nombreFemelles)	30	Terminé	
9	1ère part	Service	service/ GroupeReproductionQ ueryService.java	findAll() · getDetailGroupe(id) · prepareFormModel(Model, id)	90	Terminé	
10	1ère part	Repository	repository/ GroupeReproductionR epository.java	findAllByOrderByDateSailli eDesc · findByLotFemelleId · countByStatutIn	45	Terminé	
11	1ère part	Repository	repository/ ParametreReproductio nRaceRepository.java	findByRaceId(raceId)	15	Terminé	
12	1ère part	Model/DTO/View	model/ GroupeReproduction.j ava · ParametreReproductio nRace.java · dto/GroupeReproducti onDTO.java · GroupeReproductionD etailDTO.java · groupes/liste.jsp · form.jsp	Entités + DTO + vues création	90	Terminé	
13	Suite	Controller	controller/ ImportExportControll er.java	indexImports · showImportForm · importerExcel · exporterExcel · exporterPdf · historique	180	Terminé	
14	Suite	Service	service/ ImportExportService.j ava	importerLots/Clients/ Vaccinations/Ingredients Excel · exporterLots/Ventes/Groupes /Analyse Excel · tracerImport · tracerExport · historique	150	Terminé	
15	Suite	Repository	repository/ ImportExportReposito ry.java	byModule · byStatut · orderByDate	45	Terminé	
16	Suite	Model	model/ ImportExport.java	Entité (IMPORT/EXPORT, EXCEL/PDF, SUCCESE/ECHEC)	15	Terminé	
17	Suite	DTO	dto/ ImportExcelDTO.java · RapportFiltreDTO.jav a	DTO import	20	Terminé	
18	Suite	Vue	imports/form.jsp · imports/historique.jsp	Vues import + historique	40	Terminé	
TOTAL (min)					990		

Manoa

N°	Partie	Couche	Fichier	Fonction / Route (signature)	Durée (min)	Statut	Remarque
1	1ère part	Controller	controller/ AnalyseReproduction Controller.java	@GetMapping(/ reproduction/analyse/lots/ {lotId}) analyseLot(lotId, Model)	30	Terminé	
2	1ère part	Controller	controller/ AnalyseReproduction Controller.java	@PostMapping(/ reproduction/analyse/ generer/{lotId}) genererAnalyse(lotId, Model)	30	Terminé	
3	1ère part	Service	service/ AnalyseReproduction Service.java	analyserLot(lotId)	30	Terminé	
4	1ère part	Service	service/ AnalyseReproduction Service.java	calculerTauxAptitudeGlobal(dto) · calculerTauxRecommande(dt o) · calculerTauxFertileObserve (dto)	90	Terminé	
5	1ère part	Service	service/ AnalyseReproduction	genererDecision(dto) · enregistrerAnalyse(dto)	60	Terminé	

			Service.java				
6	1ère part	Service	service/ RepartitionReproducti veService.java	initialiserRepartitionLotFem elle(lotId) · getRepartitionByLot(lotId)	60	Terminé	
7	1ère part	Service	service/ RepartitionReproducti veService.java	calculerFemellesDisponibles (lotId)	30	Terminé	Terminé (refactorisé) : logique dans initialiserRepartitio nLotFemelle / getQuantite
8	1ère part	Service	service/ RepartitionReproducti veService.java	calculerFemellesEnCycle(lot Id)	30	Terminé	Terminé (refactorisé) : getQuantite(lotId, "EN_CYCLE")
9	1ère part	Service	service/ RepartitionReproducti veService.java	mettreAJourRepartition(lotId , statut, quantite)	30	Terminé	
10	1ère part	Repository	repository/ AnalyseReproduction LotRepository.java	findByLotIdOrderByDateAn alyseDesc(lotId)	15	Terminé	
11	1ère part	Repository	repository/ RepartitionReproducti veLotRepository.java	findByLotId · findByLotIdAndStatutRepro ductif	30	Terminé	
12	1ère part	Repository	repository/ StatutReproductifRep ository.java	findAll()	15	Terminé	
13	1ère part	Model/DTO/Vue	model/ AnalyseReproduction Lot.java · RepartitionReproducti veLot.java · StatutReproductif.java · dto/AnalyseReproduct ionLotDTO.java · RepartitionReproducti veDTO.java · analyse/analyseLot.jsp	Entités + DTO + vue analyse	90	Terminé	
14	Suite	Controller	controller/ MouvementStockCont roller.java	listMouvements · showForm · saveMouvement(ingredientI d, typeMouvement, quantite, Model)	90	Terminé	Présent (46 lignes)
15	Suite	Service	service/ MouvementStockServ ice.java	enregistrerMouvementStock · verifierStockDisponible	60	Terminé	Présent (101 lignes)
16	Suite	Service	service/ MouvementStockServ ice.java	appliquerEntree · appliquerSortie	60	Terminé	Présent : appliquerEntree (1.71) / appliquerSortie (1.80)
17	Suite	Repository	repository/ MouvementStockAlim entRepository.java	orderByDate · byIngredient	30	Terminé	Renommé : MouvementStockR epository.findAllBy OrderByldDesc()
18	Suite	Model	model/ MouvementStockAlim ent.java	Entité (ENTREE/SORTIE/AJUSTE MENT)	15	Terminé	Renommé : model/Mouvement Stock.java (le fichier existe)
19	Suite	DTO/Vue	dto/ MouvementStockDTO .java · stocks/mouvements.js p · formMouvement.jsp	DTO + vues mouvements stock	45	Terminé	
TOTAL (min)					840		

Noah

N°	Partie	Couche	Fichier	Fonction / Route (signature)	Durée (min)	Statut	Remarque
1	1ère part	Controller	controller/ DashboardController.j ava	@GetMapping(/dashboard) dashboard(Model, HttpSession)	30	Terminé	
2	1ère part	Service	service/ DashboardService.jav a	getDashboard(debut, fin)	30	Terminé	
3	1ère part	Service	service/ DashboardService.jav a	compterLotsActifs · calculerTotalPorcsActifs · compterGroupesReproductio nActifs · compterMisesBasProches	120	Terminé	
4	1ère part	Service	service/ DashboardService.jav a	calculerTauxAptitudeGlobal · calculerTauxFertiliteGlobal	60	Terminé	
5	1ère part	Service	service/ DashboardService.jav a	calculerVentesMois · calculerDepensesMois · calculerBeneficeNet	90	Terminé	

6	1ère part	Service	service/ DashboardService.java	listerStocksFaibles · listerVaccinationsAVenir(dateLimite)	60	Terminé	
7	1ère part	DTO	dto/ DashboardDTO.java	DTO indicateurs	10	Terminé	
8	1ère part	Vue	dashboard/index.jsp	Tableau de bord + graphes	20	Terminé	Graphe d'évolution ne montre pas montées/chutes + bouton générer rapport KO (correction.md)
9	1ère part	Controller	controller/ AlerteReproductionController.java	@GetMapping(/reproduction/alertes) listAlertes(Model)	30	Terminé	
10	1ère part	Controller	controller/ AlerteReproductionController.java	@PostMapping("/{id}/lire) marquerCommeLue(id)	30	Terminé	
11	1ère part	Controller	controller/ AlerteReproductionController.java	@PostMapping("/{id}/traiter) traiter(id)	30	Terminé	
12	1ère part	Service	service/ AlerteReproductionService.java	listerAlertesActives · genererAlertesMiseBasProche · marquerCommeLue · traiter	120	Terminé	
13	1ère part	Repository	repository/ AlerteReproductionRepository.java	findByStatut · findByTypeAlerteAndStatut	30	Terminé	
14	1ère part	Model/DTO/Vue	model/ AlerteReproduction.java · dto/AlerteReproductionDTO.java · reproduction/alertes/liste.jsp	Entité + DTO + vue alertes	45	Terminé	
15	Suite	Controller	controller/ IngredientController.java	listIngredients · showForm · save	90	Terminé	
16	Suite	Service	service/ IngredientService.java	findAllIngredients · creerIngredient · modifierIngredient · listerStocksFaibles	120	Terminé	
17	Suite	Repository	repository/ IngredientRepository.java	findAllByOrderByNomAsc · existsByNomIgnoreCase · findStocksFaibles	45	Terminé	
18	Suite	Model/DTO/Vue	model/Ingredient.java · dto/IngredientDTO.java · stocks/ingredients.jsp · formIngredient.jsp	Entité + DTO + vues ingrédients	60	Terminé	
TOTAL (min)					1020		

Mandresy

N°	Partie	Couche	Fichier	Fonction / Route (signature)	Durée (min)	Statut	Remarque
0	All Interface	All JSP			200	Terminé	
1	Suite	Model	model/Depense.java	Entité Dépense (categorie, dateDepense, montant, description)	15	Terminé	
2	Suite	Model	model/ CategorieDepense.java	Entité CategorieDepense (id, nom)	15	Terminé	
3	Suite	DTO	dto/DepenseDTO.java	DTO Dépense	10	Terminé	
4	Suite	DTO	dto/ CategorieDepenseDTO.java	DTO Catégorie	10	Terminé	
5	Suite	Repository	repository/ CategorieDepenseRepository.java	findAllByOrderByNomAsc · existsByNomIgnoreCase	30	Terminé	
6	Suite	Service	service/ CategorieDepenseService.java	findAllCategories · findById · créer · modifier · valider	150	Terminé	
7	Suite	BD	resources/data.sql	Catégories (Alimentation, Santé, Personnel, Transport, Maintenance, Autre)	25	Terminé	
8	Renfort	Santé	sante/* · layout/sidebar.jsp	Renfort module Santé (vaccin/vaccination/traitement/maladie) + sidebar	40	Terminé	
TOTAL (min)					495		

Fanilo

N°	Partie	Couche	Fichier	Fonction / Route (signature)	Durée (min)	Statut	Remarque
1	1ère part	Controller	controller/	@GetMapping(/lots)	30	Terminé	

			LotPorcController.java	listLots(LotFiltreDTO, Model)			
2	1ère part	Controller	controller/ LotPorcController.java	@GetMapping(/lots/form) showForm(id, Model)	30	Terminé	
3	1ère part	Controller	controller/ LotPorcController.java	@PostMapping(/lots/save) save(LotPorcDTO, Model)	30	Terminé	
4	1ère part	Controller	controller/ LotPorcController.java	@GetMapping(/lots/{id}) detail(id, Model)	30	Terminé	
5	1ère part	Controller	controller/ LotPorcController.java	@PostMapping(/lots/ archive/{id}) archiver(id)	30	Terminé	
6	1ère part	Service	service/ LotPorcService.java	rechercherLots(filtre) · prepareFormModel · creerLot · modifierLot · getDetailLot · archiverLot · existeCodeLot · verifierLotActif	240	Terminé	
7	1ère part	Repository	repository/ LotPorcRepository.java	existsByCodeLot · findByCodeLotContainingIg noreCase · findBySexe · findByObjectif · findByStatut · countByStatut	90	Terminé	
8	1ère part	Model/DTO	model/LotPorc.java · Race.java · dto/LotPorcDTO.java · LotFiltreDTO.java · LotDetailDTO.java	Entités + DTO lots	75	Terminé	
9	1ère part	Vue	lots/listeLots.jsp · formLot.jsp · detailLot.jsp	Vues lots	60	Terminé	
10	1ère part	Controller	controller/ MouvementLotContro ller.java	@GetMapping(/lots/{id}/ mouvements) listMouvements(id, Model)	30	Terminé	
11	1ère part	Controller	controller/ MouvementLotContro ller.java	@PostMapping(/lots/ mouvements/save) saveMouvement(Mouvement LotDTO, Model)	30	Terminé	
12	1ère part	Service	service/ MouvementLotServic e.java	getMouvementsByLot · enregistrerMouvement · augmenterEffectif · diminuerEffectif · verifierQuantiteDisponible	150	Terminé	
13	1ère part	Repository	repository/ MouvementLotPorcRe pository.java	findByLotIdOrderByDateMo uvementDesc(lotId)	15	Terminé	
14	1ère part	Model/DTO/Vue	model/ MouvementLotPorc.ja va · dto/MouvementLotDT O.java · lots/mouvements.jsp	Mouvements d'effectif	45	Terminé	
15	Suite	Controller	controller/ VenteController.java	listVentes · showForm · save · valider(/ventes/valider/{id}) · annuler · genererRecuPdf	180	Terminé	FICHER VIDE (0 ligne)
16	Suite	Service	service/ VenteService.java	findAll · creerVente · validerVente · annulerVente · calculerMontantTotal · genererRecuPdf	180	Terminé	FICHER VIDE (0 ligne)
17	Suite	Repository	repository/ VenteRepository.java · DetailVenteRepository .java	byDateVente · byStatut · byVenteId	45	Terminé	Présent (23 lignes)
18	Suite	Model	model/Vente.java · DetailVente.java	Entités (statuts BROUILLON/VALIDEE/A NNULEE)	30	Terminé	Présent (48 lignes)
19	Suite	DTO	dto/VenteDTO.java · DetailVenteDTO.java	DTO ventes	20	Terminé	À vérifier
20	Suite	Vue	commerce/ventes.jsp · formVente.jsp · detailVente.jsp	Vues ventes	60	Terminé	
TOTAL (min)					1400		

Nomena

N°	Partie	Couche	Fichier	Fonction / Route (signature)	Durée (min)	Statut	Remarque
1	1ère part	Controller	controller/ PeseeLotController.ja va	@GetMapping(/lots/{id}/ pesees) listPesees(id, Model)	30	Terminé	
2	1ère part	Controller	controller/ PeseeLotController.ja va	@PostMapping(/lots/ pesees/save) savePesee(PeseeLotDTO, Model)	30	Terminé	

3	1ère part	Service	service/ PeseeLotService.java	getPeseesByLot(lotId) · enregistrerPesee(dto) · calculerEvolutionPoids(lotId)	90	Terminé	
4	1ère part	Repository	repository/ PeseeLotRepository.java	findByLotIdOrderByDatePes eeAsc(lotId)	15	Terminé	
5	1ère part	Model/DTO/Vue	model/PeseeLot.java · dto/PeseeLotDTO.java · lots/pesees.jsp	Pesées	45	Terminé	
6	1ère part	Controller	controller/ VaccinController.java	/vaccins · /vaccins/form · /vaccins/save	90	Terminé	
7	1ère part	Controller	controller/ VaccinationController. java	/vaccinations · /vaccinations/form · /vaccinations/save	90	Terminé	
8	1ère part	Controller	controller/ SuiviSanitaireControll er.java	/sante/suivis · /sante/suivis/form · /sante/suivis/save	90	Terminé	
9	1ère part	Service	service/ VaccinService.java · VaccinationService.ja va · SuiviSanitaireService. java	Logique métier vaccins / vaccinations / suivis	30	Terminé	
10	1ère part	Repository	Vaccin · Vaccination · Maladie · Traitement · SuiviSanitaire Repository	Accès données santé	15	Terminé	
11	1ère part	Model	Vaccin · Vaccination · Maladie · Traitement · SuiviSanitaire	Entités santé	75	Terminé	
12	1ère part	DTO	VaccinDTO · VaccinationDTO · SuiviSanitaireDTO	DTO santé	30	Terminé	
13	1ère part	Vue	sante/vaccins · formVaccin · vaccinations · formVaccination · suivis · formSuivi	Vues santé	120	Terminé	
TOTAL (min)					750		

Davida

N°	Partie	Couche	Fichier	Fonction / Route (signature)	Durée (min)	Statut	Remarque
1	1ère part	Controller	controller/ AuthController.java	@GetMapping(/login) showLogin(Model)	30	Terminé	
2	1ère part	Controller	controller/ AuthController.java	@PostMapping(/login) login(LoginDTO, Model, HttpSession)	30	Terminé	
3	1ère part	Controller	controller/ AuthController.java	@GetMapping(/logout) logout(HttpSession)	30	Terminé	
4	1ère part	Controller	controller/ UtilisateurController.j ava	@GetMapping(/utilisateurs) listUtilisateurs(Model)	30	Terminé	
5	1ère part	Controller	controller/ UtilisateurController.j ava	@GetMapping(/utilisateurs/ form) showForm(id, Model)	30	Terminé	
6	1ère part	Controller	controller/ UtilisateurController.j ava	@PostMapping(/ utilisateurs/save) save(UtilisateurDTO, Model)	30	Terminé	
7	1ère part	Controller	controller/ UtilisateurController.j ava	@PostMapping(/ utilisateurs/desactiver/{id}) desactiver(id)	30	Terminé	
8	1ère part	Service	service/ AuthService.java	connecter(LoginDTO, HttpSession)	30	Terminé	
9	1ère part	Service	service/ AuthService.java	verifierMotDePasse(brut, hash)	30	Terminé	
10	1ère part	Service	service/ AuthService.java	deconnecter(HttpSession)	30	Terminé	
11	1ère part	Service	service/ UtilisateurService.java	findAll() · findById(id) · creer(dto) · modifier(id,dto) · desactiver(id) · emailExiste(email)	180	Terminé	
12	1ère part	Repository	repository/ UtilisateurRepository.j ava	findByEmail(email) · existsByEmail(email)	30	Terminé	
13	1ère part	Model/DTO	model/Utilisateur.java · Role.java · dto/LoginDTO.java · UtilisateurDTO.java	Entités + DTO	60	Terminé	
14	1ère part	Vue	login.jsp · utilisateurs/liste.jsp · utilisateurs/form.jsp	Connexion + liste/form utilisateurs	60	Terminé	
15	Suite	Controller	controller/ ClientController.java	@GetMapping(/clients) listClients(Model)	30	Terminé	

16	Suite	Controller	controller/ ClientController.java	@GetMapping(/clients/form) showClientForm(id, Model)	30	Terminé	
17	Suite	Controller	controller/ ClientController.java	@PostMapping(/clients/ save) saveClient(ClientDTO, Model)	30	Terminé	
18	Suite	Controller	controller/ ClientController.java	@GetMapping(/clients/{id}) detailClient(id, Model)	30	Terminé	Petit probleme de lien mais déjà réglé
19	Suite	Service	service/ ClientService.java	findAll · findById · créer · modifier · getForm	150	Terminé	
20	Suite	Repository	repository/ ClientRepository.java	findAllByOrderByNomAsc · existsByNomIgnoreCase	30	Terminé	
21	Suite	Model	model/Client.java	Entité Client	15	Terminé	
22	Suite	DTO	dto/ClientDTO.java	DTO Client	10	Terminé	
23	Suite	Vue	commerce/clients.jsp · commerce/formClient. jsp	Liste + formulaire clients	40	Terminé	
TOTAL (min)					995		

Miaro

N°	Partie	Couche	Fichier	Fonction / Route (signature)	Durée (min)	Statut	Remarque
1	1ère part	Projet	pom.xml	Configuration Maven + dépendances	20	Terminé	
2	1ère part	Projet	MadaporcApplication. java	Classe principale Spring Boot	20	Terminé	
3	1ère part	Config	application.properties	Datasource PostgreSQL + vues JSP + encodage	15	Terminé	
4	1ère part	Common	common/ AppConstants.java	Constantes globales	10	Terminé	
5	1ère part	BD	resources/schema.sql	Création des tables	25	Terminé	
6	1ère part	BD	resources/data.sql	Données de test	25	Terminé	
7	1ère part	CSS	static/css/app.css	Feuille de style	20	Terminé	
8	1ère part	Vue	layout/header.jsp	En-tête	20	Terminé	
9	1ère part	Vue	layout/sidebar.jsp	Menu latéral	20	Terminé	
10	1ère part	Vue	layout/footer.jsp	Pied de page	20	Terminé	
11	1ère part	Vue	login.jsp · placeholder.jsp	Pages globales	40	Terminé	
12	Suite	Controller	controller/ DepenseController.jav a	@GetMapping(/depenses) listDepenses(Model)	30	Terminé	
13	Suite	Controller	controller/ DepenseController.jav a	@GetMapping(/depenses/ form) showForm(id, Model)	30	Terminé	
14	Suite	Controller	controller/ DepenseController.jav a	@PostMapping(/depenses/ save) save(DepenseDTO, Model)	30	Terminé	
15	Suite	Service	service/ StockService.java	findAllDepenses()	30	Terminé	
16	Suite	Service	service/ StockService.java	enregistrerDepense(Depense DTO)	30	Terminé	
17	Suite	Repository	repository/ DepenseRepository.ja va	findAllByOrderByDateDepe nseDesc / between / byCategorie / totalDepenses	15	Terminé	
18	Suite	Vue	finance/depenses.jsp · finance/formDepense.j sp	Liste + formulaire dépenses	40	Terminé	
TOTAL (min)					440		

La conception en plus de la réalisation du projet a mis 19 jours soit environ 3 semaines.

1.3) Budget:

Membre	Rôle	Temps(min)	Temps(heures)	Tarif horaire	Budget Total
Miaro	Dev Junior	440	7.33	10000	73 333
Davida	Dev Junior	995	16.58	10000	165 833
Fanilo	Dev Junior	1400	23.33	10000	233 333
Sarobidy	Dev Junior	750	12.5	10000	125 000
Tsanta	Dev Junior	990	16.5	10000	165 000

Mickaella	Dev Junior	760	12.67	10000	126 667
Manoa	Dev Junior	840	14	10000	140 000
Noah	Dev Junior	1020	17	10000	170 000

En sommant le salaire de chaque développeur junior, on obtient donc la somme de **1 199 166 Ar**

2) Perspectives du projet:

Le projet MADAPORC a été conçu pour répondre au besoin critique de structuration de la filière porcine à travers une gestion technique précise et une analyse reproductive automatisée. Si la version actuelle pose les fondations nécessaires à la gestion de cycle, de santé et de traçabilité, elle offre un potentiel d'évolution significatif pour s'adapter aux réalités du terrain et aux avancées technologiques.

. Évolutivité technologique (Scalabilité)

Bien que l'architecture actuelle repose sur une pile robuste (Spring Boot MVC, JPA, PostgreSQL), nous envisageons pour les versions futures :

- Transition vers une architecture RESTful API : Passer progressivement à une architecture découplée, permettant une meilleure réactivité et une séparation nette entre la logique métier et la couche présentation (type React ou Vue.js).
- Optimisation des données : À mesure que le volume d'historisation (pesées, suivis sanitaires) augmentera, nous prévoyons l'implémentation de solutions de mise en cache (Redis) pour garantir la fluidité du dashboard et des rapports générés.

. Intelligence Artificielle et Aide à la Décision

L'analyse reproductive développée constitue le socle d'une future intelligence métier plus poussée :

- Modèles prédictifs : Intégrer des algorithmes de Machine Learning pour prédire les cycles de chaleurs et optimiser le taux de fertilité, en se basant sur l'historique des données collectées (taux de survie des porcelets, performance des races).
- Automatisation des alertes : Transformer le système d'alertes actuel en un moteur de notification intelligent capable d'envoyer des alertes proactives avant même l'apparition d'une anomalie sanitaire détectée via les suivis de poids.

. Intégration Écosystémique

- **Couplage GIS** : En tirant profit des systèmes d'information géographique, nous pourrions cartographier les fermes sur une plateforme dynamique pour visualiser les flux de cheptel et les zones à risque sanitaire, offrant ainsi une vision spatiale en plus de la gestion statistique.
- **Interopérabilité** : Faciliter l'échange de données avec d'autres systèmes de gestion agricole (via Webhooks) pour s'intégrer dans un écosystème agricole national plus large, favorisant la traçabilité complète de la chaîne de valeur.

Conclusion:

Le projet MADAPORC répond à un besoin concret et pressant du secteur porcin à Madagascar : remplacer une gestion artisanale, source d'erreurs et de pertes de productivité, par un outil numérique fiable, structuré et évolutif. En s'appuyant sur une architecture robuste (Java/Spring Boot, PostgreSQL) et une conception modulaire centrée sur les règles métier, l'application pose les bases d'une gestion technique rigoureuse des lots, de la reproduction, de la santé et des flux financiers.

Au-delà de la simple digitalisation, MADAPORC se veut un véritable levier d'aide à la décision, grâce à des indicateurs automatisés et une traçabilité complète. Les perspectives d'évolution — API REST, intelligence artificielle prédictive, intégration géospatiale — ouvrent la voie à un outil durable, capable de s'adapter aux mutations du secteur et d'accompagner la montée en compétences des éleveurs et des exploitants.

Ainsi, MADAPORC ne se limite pas à une application de gestion : il incarne une vision moderne, intégrée et responsable de l'élevage porcin à Madagascar.

Synthèse de présence de chaque membre tout au long du projet:

Membre	Nb absence	Date absence	Cause	Point
Tsanta	2	02/07/2026 et 01/07/2026	malade	1
Faniry	Jamais présent	0	injustifié	-3

Fanilo	0	0	Toujours présent	1
Noah	0	0	Toujours présent	1
Sarobidy	2	20/06/2026	Toujours présent	1
Davida	1	13/06/2026	malade	1
Mandresy	1	04/07/2026	Toujours présent	1
Manoa	0	0	Toujours présent	1
Irina	Jamais présent	0	injustifié	-3